

BEAGLEBONE AI ROBOT

System Description, ROS Architecture, and
Navigation and Mapping Algorithms

Jason Stumfol

Repository: <https://github.com/jstumfol12/bbai-robot>

State of the robot as of 2 October 2026

*Sections marked **In flux** describe configuration that was being changed while this document was written. The repository is the authority for anything that disagrees with this text.*

Contents

List of Figures	4
List of Tables	5
1 SYSTEM OVERVIEW	6
1.1 PURPOSE AND SCOPE	6
1.2 GOALS	6
1.3 PLATFORM SUMMARY	7
1.4 DIVISION OF WORK BETWEEN THE COMPUTERS	7
1.5 STATUS	9
2 HARDWARE BRING-UP	10
2.1 OPERATING SYSTEM AND BOOT	10
2.2 DEVICE TREE	10
2.3 SERIAL PORTS	11
2.4 LIBROBOTCONTROL FORK	12
2.5 MOTOR 1 DIRECTION FIRMWARE ON PRU2_0	12
2.6 WHEEL ENCODERS	12
2.7 INERTIAL MEASUREMENT UNIT	13
2.8 LIDAR	13
2.9 GPS	14
2.10 BLUETOOTH	14
2.11 OAK-D-LITE CAMERA	14
2.12 POWER	15
3 ROS ARCHITECTURE	16
3.1 NETWORK AND MASTER	16
3.2 PACKAGES AND WORKSPACES	16
3.3 NODES AND TOPICS	17
3.4 COORDINATE FRAMES	18
3.5 BRINGUP SEQUENCE	19

3.6	BASE NODE	19
3.7	PS4 TELEOP	19
3.8	CHASE NODE	20
4	DRIVE KINEMATICS AND WHEEL ODOMETRY	22
4.1	NOTATION	22
4.2	FROM ENCODER COUNTS TO WHEEL TRAVEL	22
4.3	SKID-STEER KINEMATICS	23
4.4	ODOMETRY INTEGRATION	23
4.4.1	Odometry uncertainty	24
4.5	FROM VELOCITY COMMAND TO MOTOR DUTY	25
4.6	WHY SKID-STEER TURNING NEEDS SO MUCH POWER	25
5	ATTITUDE AND HEADING	27
5.1	SENSOR FRAME	27
5.2	SENSOR ERROR MODEL	27
5.3	GYROSCOPE HEADING	28
5.3.1	Bias removal at start-up	28
5.3.2	Heading error growth	28
5.3.3	Zero-velocity update	28
5.4	TILT FROM THE ACCELEROMETER	29
5.5	MAGNETOMETER COMPASS	29
5.5.1	Measurement model	29
5.5.2	Planar calibration	30
5.5.3	Heading from the field	31
5.5.4	Tilt compensation	31
5.5.5	Limits indoors	31
5.6	FUSING GYRO AND COMPASS	32
6	GPS MEASUREMENT MODEL	33
6.1	NMEA TO NAVSATFIX	33
6.2	GEODETIC TO LOCAL COORDINATES	34
6.3	THE NAVSAT TRANSFORM	34
7	SENSOR FUSION: THE GPS EKF	36
7.1	THE DISCRETE EKF	36
7.2	STATE AND PROCESS MODEL	36
7.3	MEASUREMENTS	37
7.4	OBSERVABILITY AND BEHAVIOUR WITHOUT A FIX	38
7.5	RELATION TO THE SLAM MAP	38

8	MAPPING: LIDAR SLAM	40
8.1	OCCUPANCY GRID	40
8.2	GMAPPING	40
8.2.1	Rao–Blackwellized particle filter	40
8.2.2	When scans are processed	41
8.2.3	Parameters	42
8.3	HECTOR MAPPING	42
8.4	SCAN RATE AND ROTATION	43
8.5	DIAGNOSING THE SLAM DRIFT OF 1 OCTOBER	43
9	OPERATING MODES	45
9.1	LOCALIZATION AND MAPPING MODES	45
9.2	WHAT HAPPENS WHEN A SENSOR IS MISSING	46
9.3	DRIVING MODES	46
10	KNOWN ISSUES AND NEXT STEPS	47
10.1	OPEN ISSUES	47
10.2	SUGGESTED IMPROVEMENTS	47
A	REFERENCE	49
A.1	REPOSITORY MAP	49
A.2	BASE NODE PARAMETERS	49
A.3	SYMBOLS	49
A.4	BUILDING THIS DOCUMENT	49
	Bibliography	52

List of Figures

1.1	Physical and network connections	8
3.1	ROS computation graph (gmapping mode with GPS fusion)	17
3.2	Transform tree	18

List of Tables

1.1	Hardware	7
1.2	Subsystem status, 2 October 2026	8
2.1	Device tree changes on top of upstream	11
2.2	UART assignments (confirmed on the board)	11
2.3	Encoder and motor channel mapping (spin and lifted-wheel tests, 2026-10-01)	13
3.1	Catkin packages	16
3.2	Topics (message types without their package prefix)	17
3.3	PS4 controller mapping (hid-sony , kernel 4.14)	20
4.1	Geometry and drive parameters	22
4.2	Commanded duty for typical commands	25
7.1	EKF inputs (gps_ekf.yaml)	37
8.1	gmapping parameters (laptop/bbai_slam/gmapping_bbai.launch)	42
9.1	Localization and mapping modes	45
9.2	Degradation by sensor	46
A.1	Where things are in jstumfoll2/bbai-robot	49
A.2	base.yaml (branch slam-estimator-updates)	50
A.3	Main symbols	50

Chapter 1

SYSTEM OVERVIEW

1.1 PURPOSE AND SCOPE

This document records how the BeagleBone AI robot is built and how its software fits together: every change that was needed to bring each device up on the board, how the devices, computers and ROS nodes are connected, and the mathematics of the navigation and mapping algorithms. The navigation chapters (Chapters 4–8) derive the wheel odometry, the attitude and heading estimate from the gyroscope and magnetometer, the GPS measurement model, the extended Kalman filter that fuses them, and the SLAM algorithms that build the map. Chapter 9 lists the operating modes that are available with or without each sensor, and Chapter 10 collects known problems, including the analysis of the SLAM drift observed on 1 October 2026.

The notation and the sensor-error models are taken from the author’s master’s thesis on the two-wheeled inverted pendulum (TWIP) [1], in particular its treatment of IMU measurements (thesis §4.3.1), its sensor noise model with a constant and a walking bias (thesis §7.5), and its discrete Kalman filter (thesis §7.7). Where this document reuses a result from the thesis it says so, so that the derivations can be read side by side.

1.2 GOALS

The goals, in Jason’s priority order as of 1 October 2026, are:

1. Back up and version all of the board’s work (done: this repository and the `librobotcontrol` fork).
2. Chase the dog using on-camera detection on the OAK-D-Lite.
3. Map the house with lidar SLAM.
4. Drive manually with a PS4 controller over Bluetooth.
5. Publish the IMU, wheel odometry and GPS as ROS topics and fuse them into a single navigation state.

1.3 PLATFORM SUMMARY

The robot is a small four-wheel skid-steer chassis. All four wheels are driven, each by its own DC gear motor with a Hall-effect encoder. A BeagleBone AI with a Robotics Cape is the only on-board computer. Table 1.1 lists the hardware and Figure 1.1 shows how it is connected.

Table 1.1: Hardware

Item	Details
Computer	BeagleBone AI (TI AM5729: 2× Cortex-A15 at 1.5 GHz, 4× PRU, 2× C66x DSP, 4× EVE). Debian 10 Buster, kernel 4.14.108-ti-r136, Python 3.7, ROS Melodic built from source.
Cape	Robotics Cape (Beaglebone Blue functionality as a cape): 4 H-bridge motor channels, 4 encoder inputs, MPU-9250 IMU, barometer, UART and I ² C headers, 2-cell LiPo input and charger.
Drive	4-wheel skid steer. Wheel diameter 79.2 mm, left-right wheel centres 120 mm, axle spacing 101 mm.
Encoders	DAGU 8-pole magnet rings read by Allegro A1230 dual Hall sensors (open-drain outputs). Measured 15 counts per wheel revolution.
IMU	InvenSense MPU-9250 on the cape: 3-axis gyroscope, accelerometer and AK8963 magnetometer.
Lidar	SLAMTEC RPLIDAR A1M8 on a cape UART (/dev/ttyS3), about 7.2 Hz scan rate, 0.15 m to 12 m range.
GPS	GlobalSat EM-506 (SiRF Star IV) on /dev/ttyS2, NMEA at 4800 baud, 1 Hz.
Camera	Luxonis OAK-D-Lite (stereo depth plus RGB, Myriad X VPU) on the BeagleBone’s USB-A port.
Radio	On-board Wi-Fi and Bluetooth (AzureWave AW-CM256SM, Broadcom BCM43455); Bluetooth HCI on UART6 (/dev/ttyS5).
Controller	Sony DualShock 4 (PS4) over Bluetooth.
Power	2-cell LiPo pack (8.17 V full) into the cape, or a bench supply.
Off-board	Ubuntu 20.04 laptop with ROS Noetic (SLAM, sensor fusion, RViz); Windows PC (development, camera calibration, board serial console on COM3).

1.4 DIVISION OF WORK BETWEEN THE COMPUTERS

The board runs everything that touches hardware: the ROS master, the lidar driver, the motor, encoder and IMU node, the GPS node, the PS4 teleop node and the chase node. The OAK-D-Lite runs its neural network on its own Myriad X, so the board only receives detections. The laptop runs everything that is heavy but not time-critical: SLAM, the EKF and navsat transform, and RViz. The laptop joins the board’s ROS master over Wi-Fi (ROS_MASTER_URI=<http://192.168.3.120:11311>).

This split follows a decision made on 1 October 2026: keep everything on the BeagleBone where possible, run SLAM on the laptop, and treat a Raspberry Pi 5 as the fallback compute if the board runs out of headroom.

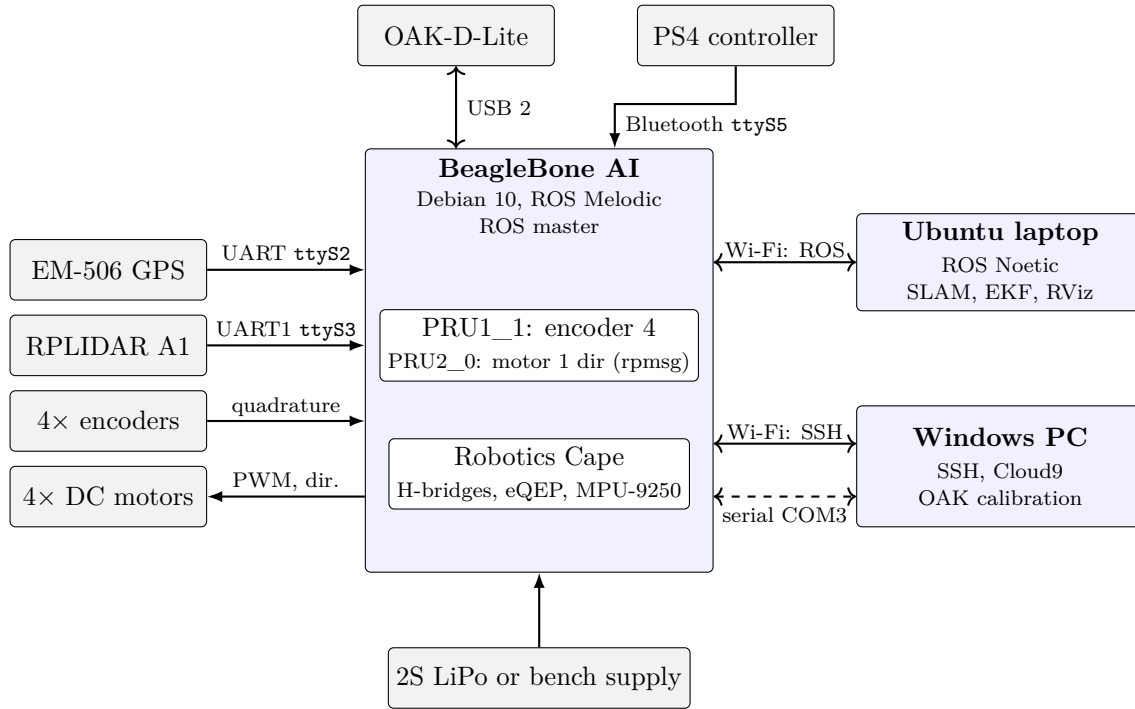


Figure 1.1: Physical and network connections

Table 1.2: Subsystem status, 2 October 2026

Subsystem	State	Notes
Motors (all four)	Working	Motor 1 needs the PRU2_0 firmware (§2.5).
Encoders (all four)	Working	Needed pull-ups in the device tree (§2.2).
Wheel odometry	Working	0.914 m measured for a 0.914 m push.
IMU (gyro, accel)	Working	Heading drift was 1.6 °/min parked; zero with the new ZUPT.
Magnetometer	Calibrated	Compass agrees with the gyro to 3% over two turns.
GPS	Working	No fix indoors, as expected.
Lidar	Working	7.2 Hz on <code>ttyS3</code> .
Bluetooth + PS4	Working	Report stalls of 100–500 ms in bursts.
SLAM (gmapping)	Poor	Loses track when driving; drifted when stationary. Fixes being tested (§8.5).
GPS fusion (EKF)	Built, untested	Needs an outdoor run with a fix.
OAK-D-Lite detection	Working	Resets the board under full load; waiting on separate camera power.
Chase controller	Dry run only	Waiting on camera power.

1.5 STATUS

In flux. SLAM tuning, the EKF configuration and the teleop turn curve were all being changed on 2 October 2026. Each is marked where it appears.

Chapter 2

HARDWARE BRING-UP

This chapter records every change that was needed to make each device work on the BeagleBone AI. The Robotics Cape was designed for the BeagleBone Black and Blue, whose AM335x processor has a different pin multiplexer, different GPIO numbering and different peripheral addresses from the AM5729 on the BeagleBone AI. Almost every problem in this chapter comes from that mismatch.

2.1 OPERATING SYSTEM AND BOOT

The board runs the 2020-04-06 TI image (Debian 10 Buster, kernel 4.14.108-ti-r136). Debian 10 is out of support, so `/etc/apt/sources.list` points at archive.debian.org with the security line commented out; without that, `apt` cannot install anything (it was needed for `libatlas-base-dev`, which `depthai` uses).

The boot configuration is `system/uEnv.txt`, installed as `/boot/uEnv.txt`:

```
uname_r=4.14.108-ti-r136
dtb=am5729-beagleboneai-roboticscape.dtb
cmdline=coherent_pool=1M
```

The `dtb=` line makes U-Boot load the custom Robotics Cape device tree instead of the stock one. Journald was made persistent (`/var/log/journal`) on 1 October 2026 so that the cause of an unexpected reboot survives the reboot.

2.2 DEVICE TREE

The device tree is built from `beagleboard/BeagleBoard-DeviceTrees` (branch `v4.14.x-ti`, commit `4a9c0a6`) plus `devicetree/roboticscape-bbai.patch`. The full source files are also kept in `devicetree/`. Table 2.1 lists every change and why it was made.

The patch also moves the `epwmss2/ehrpwm2` and UART nodes around without changing what they enable, and adds a model string.

Table 2.1: Device tree changes on top of upstream

Change	Reason
<code>&hdm</code> disabled	HDMI shares pins with cape signals.
<code>&i2c4</code> enabled on P9.19/P9.20 (mode 7), shared balls set to GPIO input	Cape I ² C2 header. The BB-AI routes two SoC balls to each of these header pins, so the unused ball must be parked.
P9.13 muxed to <code>pr2_pru0_gpo15</code> (mode 13)	Motor 1 direction B. The AM335x GPIO this pin used is not available on the AM5729 ball that reaches the header; the only usable function is a PRU output (§2.5).
<code>PIN_INPUT_PULLUP</code> on all 8 encoder pins (eQEP1A/B, eQEP2A/B, eQEP3A/B, PRU1 gpi16/18)	The A1230 outputs are open-drain and need a pull-up. Without it no encoder counted at all (2026-10-01).
<code>&uart6</code> enabled	Bluetooth HCI for the BCM43455. It had been commented out in the base <code>.dts</code> in 2022 for an unrecorded reason; re-enabled and tested on 2026-10-01.

Build and install. On the board (needs `sudo` and a reboot):

```
cd ~/src/BeagleBoard-DeviceTrees && git apply .../roboticscape-bbai.patch
make src/arm/am5729-beagleboneai-roboticscape.dtb
sudo cp /boot/dtbs/4.14.108-ti-r136/am5729-beagleboneai-roboticscape.dtb ~/roboticscape.dtb.bak
sudo cp src/arm/am5729-beagleboneai-roboticscape.dtb /boot/dtbs/4.14.108-ti-r136/
sudo reboot
```

The backup in the home folder is the rollback path; the board’s eMMC can also be repaired from a microSD boot if the new tree does not boot.

2.3 SERIAL PORTS

The cape’s UART header names, the AM5729 UART instances and the Linux device names do not line up, which caused the lidar to be looked for on the wrong port at first. Table 2.2 gives the confirmed mapping.

Table 2.2: UART assignments (confirmed on the board)

Device	Cape header	Linux device	Settings
RPLIDAR A1	UART1	<code>/dev/ttyS3</code> (old name <code>ttty03</code>)	115 200 baud
EM-506 GPS	GPS	<code>/dev/ttyS2</code> (SoC UART3)	4800 baud 8N1
Bluetooth HCI	(on board)	<code>/dev/ttyS5</code> (SoC UART6)	3 Mbaud, RTS/CTS
Debug console	(on board)	<code>tttyS0</code>	115 200 baud, PC COM3

2.4 LIBROBOTCONTROL FORK

Motors, encoders and the IMU are driven through `librobotcontrol` (used from Python via `rcpy`). The upstream library supports the BeagleBone AI only partially. The fork `jstum-foll2/librobotcontrol`, branch `v1.1_AIfixes`, is a submodule of this repository and carries two commits on top of upstream `99a08db`: Jason’s 2022 work (`d90e24b`) and the uncommitted board changes recovered on 2026-10-01 (`593182b`). The functional changes are:

- **GPIO numbering** (`motor.c`). All motor direction pins and the motor standby pin were remapped to AM5729 GPIO banks (for example MDIR1A is `gpio5_0`, MOT_STBY is `gpio6_20`). The per-channel polarity table was changed to $\{+1, -1, +1, -1\}$.
- **A virtual GPIO chip for the PRU** (`gpio.c`). Chip number 11 is redirected to the character device `/dev/rpmsg_pru32`. Setting a pin on chip 11 writes the ASCII character ‘0’ or ‘1’ to that device instead of issuing a GPIO ioctl. Motor 1’s direction B pin is defined as chip 11, so `rc_motor_init()` fails unless the PRU firmware below is running.
- **eQEP sysfs paths** (`encoder_eqep.c`). Updated to the AM5729 addresses (`4843e000.epwmss` and so on).
- **IMU bus** (`mpu.c`). The MPU-9250 is on I²C bus 3 with its interrupt on `gpio5_17` (P9.25).
- **PRU encoder** (`encoder_pru.c`). Encoder channel 4 is counted by firmware on PRU1_1 (`am57xx-pru1_1-rc-encoder-fw-c`), read from PRU shared memory at offset 0.

2.5 MOTOR 1 DIRECTION FIRMWARE ON PRU2_0

The PRU firmware `gpio_rpmsg.pru2_0.c` (`librobotcontrol/pru_firmware/customfw/`) is about 100 lines. It opens an RPMsg channel named `rpmsg-pru` on port 32, which the kernel exposes as `/dev/rpmsg_pru32`. Each message received is parsed with `atoi`; a non-zero value sets bit 15 of the PRU output register R30 (P9.13), and zero clears it. After each message the firmware waits $10\,200 + 4\,000\,000$ PRU cycles, about 20 ms at 200 MHz, so motor 1 can reverse direction at most about 50 times per second. That is far faster than any command the robot sends.

The firmware is installed as `/lib/firmware/am57xx-pru2_0-fw`. It is not started at boot; `base_node.py` starts it before initialising the motors, equivalent to

```
echo am57xx-pru2_0-fw > /sys/class/remoteproc/remoteproc2/firmware
echo start > /sys/class/remoteproc/remoteproc2/state
```

and then waits up to 5 s for `/dev/rpmsg_pru32` to appear.

2.6 WHEEL ENCODERS

Each wheel has a DAGU 8-pole magnet ring (4 north, 4 south) read by an Allegro A1230 dual-channel Hall sensor in quadrature. Two facts from the datasheets explained why nothing counted at first:

- The A1230 outputs are open-drain, and the datasheet requires pull-up resistors. The cape has none on these lines, so the internal pull-ups were enabled in the device tree (Table 2.1).
- The A1230 needs at least 3.3 V; the cape’s encoder headers supply exactly 3.3 V, which is in specification but at the bottom of the range.

An 8-pole ring with a quadrature sensor gives $4 \times 4 = 16$ edges per revolution in principle. A 3 ft (0.914 m) push measured about 55 counts on every wheel, all four within 2 counts of each other, giving 15.0 counts per revolution, and that value is in `base.yaml`.

Table 2.3: Encoder and motor channel mapping (spin and lifted-wheel tests, 2026-10-01)

Wheel	Encoder channel	Counter	Encoder sign	Motor channel
Left back	1	eQEP	+1	M1
Right back	2	eQEP	−1	M4
Left front	3	eQEP	+1	M2
Right front	4	PRU1_1	−1	M3

All four motors drive forward with positive duty (`motor_signs` all +1), after the library’s own polarity table. The right-side encoders count down when driving forward because they are mounted mirrored, which the encoder signs correct.

2.7 INERTIAL MEASUREMENT UNIT

The MPU-9250 is read through `rcpy.mpu9250` with the magnetometer enabled. `rcpy` returns acceleration in m/s^2 , angular rate in $^\circ/\text{s}$ and magnetic field in μT ; `base_node.py` converts to SI (rad/s, T) before publishing. Relevant datasheet values at the default $\pm 250^\circ/\text{s}$ range are a rate-noise spectral density of $0.01^\circ/\text{s}/\sqrt{\text{Hz}}$, an initial zero-rate offset of up to $\pm 5^\circ/\text{s}$, and a zero-rate offset that can move by up to $\pm 30^\circ/\text{s}$ over the full temperature range. The last figure is why the gyroscope bias must be measured at every start (§5.3).

The magnetometer read about $150\mu\text{T}$ on the robot, against Earth’s roughly $50\mu\text{T}$. The extra field comes from the motor magnets, the encoder magnet rings and current in the wiring, which is why it needs the calibration in §5.5.

2.8 LIDAR

The RPLIDAR A1M8 is wired to the cape’s UART1 header and appears as `/dev/ttyS3`. It reports healthy and publishes `/scan` at about 7.2 Hz through `rplidar_ros` (catkin workspace `~/melodic-catkin-ws`). The A1 scans clockwise; the driver converts to the counter-clockwise ROS convention.

2.9 GPS

The EM-506 is a SiRF Star IV receiver that outputs NMEA 0183 at 4800 baud on `/dev/ttyS2`. Two pieces of software read it:

- `gps/libgps-bbai-ttyS2-4800.patch`: a patch to the small C library `wdalmut/libgps` that changes its port and baud rate. This was the 2022 test path.
- `ros/bbai_base/scripts/gps_node.py`: the ROS publisher used now (§6.1). It configures the UART itself with `termios`.

Indoors the receiver reports no fix; a fix needs open sky.

2.10 BLUETOOTH

The on-board BCM43455 Bluetooth controller talks HCI over SoC UART6. Three things were needed to bring it up:

1. `&uart6` enabled in the device tree (Table 2.1). It then appears as `/dev/ttyS5`.
2. `system/blacklist-btsdio.conf` in `/etc/modprobe.d/`. The `btsdio` driver otherwise creates a non-working `hci0` on the SDIO bus, so the real controller would become `hci1`.
3. `ros/bbai_teleop/bt-up.sh`, run at boot by `bbai-bluetooth.service`. It finds the `tty` whose device is `48068000.serial`, attaches it with `hciattach <tty> bcm43xx 3000000 flow` (retrying at 3Mbaud if the chip was already initialised), brings `hci0` up, disables sniff mode on new links and turns off Wi-Fi power saving. The last two keep the controller’s input reports at full rate, because the BCM43455 shares one radio between Wi-Fi and Bluetooth.

The PS4 controller is paired once with `pair-ps4.sh`, which runs the whole `bluetoothctl` conversation (agent, scan, pair, trust, connect) in a single session. The kernel’s `hid-sony` driver then creates `/dev/input/js0`.

2.11 OAK-D-LITE CAMERA

The camera runs with `depthai 2.19.1` on the board’s Python 3.7 over USB 2, at about 15 fps for RGB plus depth. `MobileNet-SSD` runs on the camera’s Myriad X and detects the dog at 93–99% confidence, about 10 detections per second. The camera’s EEPROM calibration was found blank; it was self-calibrated on the PC with `depthai v2` tools and flashed on 2026-10-01 (stereo reprojection error 0.35, baseline 7.43 cm). The procedure is in `docs/CALIBRATION.md` and the backup is in `camera/pc-tools/`.

2.12 POWER

The board reset several times on 1 October 2026. Persistent journald, a once-per-second battery logger (`~/power.log`) and the serial console narrowed the cause down:

- The battery input held steady (7.7 V to 8.2 V) through resets.
- With the motor driver and lidar stopped, the camera ran detection for 90 s with no reset.
- The console showed the camera's USB disconnecting and reconnecting immediately before a reset, with no kernel panic.

The conclusion is that the cape's 5 V rail, which feeds the BeagleBone and its USB port, drops out when the camera (1 A to 1.5 A), motors, lidar and Bluetooth all run together. The fix in progress is to power the camera from a separate battery bank through a USB splitter cable.

Chapter 3

ROS ARCHITECTURE

3.1 NETWORK AND MASTER

ROS 1 is used throughout: Melodic on the board (built from source into `/opt/ros/melodic`) and Noetic on the laptop. The two are compatible on the wire for every message type used here. The board runs the ROS master, so the board must be brought up first. Both machines set

```
ROS_MASTER_URI=http://192.168.3.120:11311
ROS_IP=<own address> # 192.168.3.120 on the board, 192.168.3.147 on the laptop
```

ROS_IP matters because ROS nodes advertise their own address to the master; without it the laptop would advertise a hostname the board cannot resolve. On the laptop the variables are set by `laptop/bbai_slam/env.sh`.

3.2 PACKAGES AND WORKSPACES

Table 3.1: Catkin packages

Package	Workspace	Contents
bbai_base	~/melodic-catkin-ws (board)	base_node.py (motors, encoders, IMU), gps_node.py, mag_calibrate.py, motor_map_ test.py, robot.launch, base.yaml
rplidar_ros	~/melodic-catkin-ws (board)	SLAMTEC lidar driver
bbai_teleop	~/teleop_ws (board)	ps4_teleop.py, teleop.launch, Bluetooth scripts
bbai_chase	~/chase_ws (board)	chase_node.py, chase.launch
bbai_slam (scripts)	~/bbai_slam (laptop)	gmapping, hector and lidar-only launch files, GPS fusion launch, RViz config, start and reset scripts

Python 3 workspaces on the board are built with

```
catkin_make -DPYTHON_EXECUTABLE=/usr/bin/python3
```

3.3 NODES AND TOPICS

Figure 3.1 shows the computation graph and Table 3.2 lists every topic.

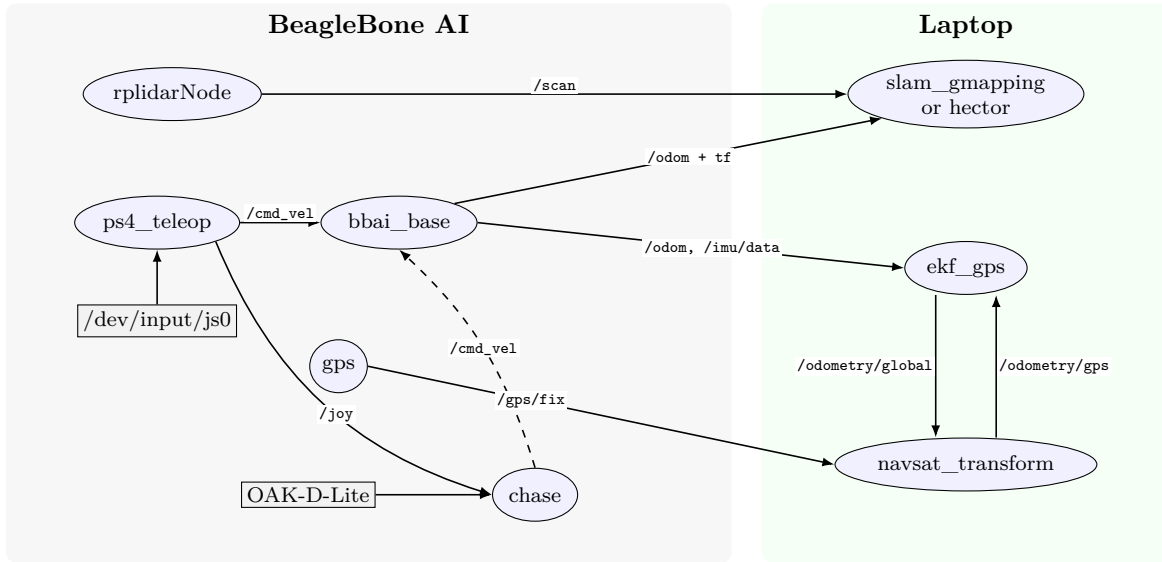


Figure 3.1: ROS computation graph (gmapping mode with GPS fusion)

Table 3.2: Topics (message types without their package prefix)

Topic	Type	Rate	Publisher → subscribers
<code>/scan</code>	LaserScan	7.2 Hz	rplidarNode → SLAM, RViz
<code>/odom</code>	Odometry	30 Hz	bbai_base → EKF, RViz
<code>/imu/data_raw</code>	Imu	30 Hz	bbai_base (rates and acceleration, no orientation)
<code>/imu/data</code>	Imu	30 Hz	bbai_base (adds compass yaw) → EKF, navsat_transform
<code>/imu/mag</code>	MagneticField	30 Hz	bbai_base (raw field, tesla)
<code>/wheel_ticks</code>	Int32MultiArray	30 Hz	bbai_base (raw counts of channels 1–4, debugging)
<code>/cmd_vel</code>	Twist	20–100 Hz	ps4_teleop, chase → bbai_base
<code>/joy</code>	Joy	20–100 Hz	ps4_teleop → chase
<code>/gps/fix</code>	NavSatFix	1 Hz	gps → navsat_transform
<code>/gps/vel</code>	TwistStamped	1 Hz	gps (ENU ground velocity, only with a fix)
<code>/odometry/global</code>	Odometry	20 Hz	ekf_gps (frame <code>gps_map</code>) → navsat_transform, user
<code>/odometry/gps</code>	Odometry	1 Hz	navsat_transform → ekf_gps

Topic	Type	Rate	Publisher → subscribers
/gps/filtered	NavSatFix	10 Hz	navsat_transform (fused estimate as latitude/longitude)
/map	OccupancyGrid	0.5 Hz	SLAM → RViz, map_saver
/chase/target	PointStamped	15 Hz	chase (target in the camera frame)
/chase/status	String	2 Hz	chase (human-readable state)
/chase/cmd_vel_ preview	Twist	15 Hz	chase (what it would command)

In flux. /imu/data, the GPS fusion launch file and its EKF configuration are on branch `slam-estimator-updates` (commit 5d4c6b6) and not yet merged into `main`. This document describes that branch.

3.4 COORDINATE FRAMES

Frames follow ROS REP 103 and REP 105 [2, 3]: right-handed, x forward, y left, z up on the body, and east-north-up (ENU) for world frames, with yaw ψ measured counter-clockwise from the world x axis. REP 105 defines three world-fixed frames with different guarantees:

odom continuous (no jumps) but drifts without bound. Published by `bbai_base` from wheel odometry and the gyroscope.

map does not drift, but may jump when a correction arrives. Published by SLAM as the transform `map→odom`.

earth/UTM the GPS frame, related to **map** by the navsat transform (§6.3).

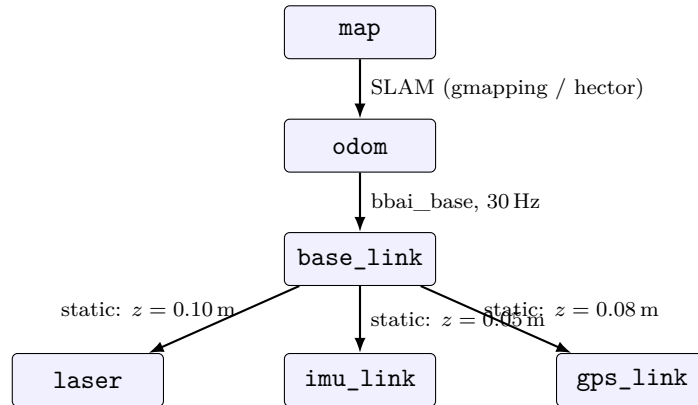


Figure 3.2: Transform tree

The three static transforms in `robot.launch` have zero horizontal offset and zero rotation. They were entered as placeholders and have not been measured. The lidar’s yaw in particular must match how the A1 is physically mounted, which matters for SLAM (§8.5).

Each transform in the tree must have exactly one publisher. In particular, only one node may

publish odom→base_link: either bbai_base (publish_tf: true, the current setting) or an EKF, never both.

3.5 BRINGUP SEQUENCE

1. Board: `roslaunch bbai_base robot.launch`. This starts the master, `rplidar_ros`, `bbai_base` (which measures the gyro bias for 2s, so the robot must be still), `gps_node`, and the three static transforms. Arguments `lidar:=false` and `gps:=false` leave those devices out. `bbai_base` is respawned automatically 5s after it exits.
2. Board: `roslaunch bbai_teleop teleop.launch`.
3. Laptop: `~/bbai_slam/start_slam.sh [gmapping|hector|lidar_only]` (gmapping by default), which also opens RViz.
4. Laptop, optional: `roslaunch ~/bbai_slam/gps_fusion.launch`.
5. Board, optional: `roslaunch bbai_chase chase.launch` (dry run unless `dry_run:=false`).

The full command list is `docs/COMMANDS.md`.

3.6 BASE NODE

`base_node.py` runs a fixed-rate loop at 30 Hz. Each cycle it reads the IMU and the four encoders, integrates the pose (Chapter 4), publishes `/odom`, the `odom→base_link` transform, `/imu/data_raw`, `/imu/mag` and `/wheel_ticks`, and enforces the command timeout: if no `/cmd_vel` message has arrived for 0.5s, all motors are set to zero. Motors are also zeroed when the node exits.

With `mag_heading: true` it also publishes `/imu/data`: the same rates and accelerations as `/imu/data_raw` plus an orientation whose yaw comes from the calibrated magnetometer (§5.5). While the robot is parked it applies a zero-velocity update to the heading and the gyro bias (§5.3.3).

In flux. The compass output, the zero-velocity update and `mag_calibrate.py` are on branch `slam-estimator-updates`; the version on `main` has neither.

3.7 PS4 TELEOP

`ps4_teleop.py` reads the Linux joystick API directly (`/dev/input/js*`, 8-byte events of time, value, type and number) and looks the device up by name (`Wireless Controller`), skipping the controller’s separate motion-sensor device. It publishes `/joy` and, only while the deadman button is held, `/cmd_vel`. Publishing runs as a 20 Hz heartbeat that speeds up to at most 100 Hz while the sticks move, and all queued events are drained on each read so the commands never lag behind a backlog.

The stick values are $a \in [-1, 1]$ after dividing by 32767, with a dead zone $|a| < 0.08$ mapped to zero. Stick “up” is negative on the joystick API, so both axes are negated. In the repository version

Table 3.3: PS4 controller mapping (`hid-sony`, kernel 4.14)

Input	Index	Effect
L1 (hold)	button 4	Deadman. Nothing moves unless held; release sends one zero Twist
Left stick vertical	axis 1	<code>linear.x</code> , 0.3 m/s full scale (turbo 0.7 m/s)
Right stick horizontal	axis 3	<code>angular.z</code> (curve below)
R1 (with L1)	button 5	Turbo scaling
R2 (L1 released)	button 7	Arms the chase node

the command is linear in the stick,

$$v = -k_v a_1, \quad \omega = -k_\omega a_3, \quad (3.1)$$

with $(k_v, k_\omega) = (0.3, 1.5)$ normally and $(0.7, 8.0)$ in turbo.

In flux. On 2 October 2026 the turn axis was changed to a curve because 1.5 rad/s at full stick could not overcome skid-steer friction (§4.6). The installed version gives about 1.5 rad/s at half stick, 6 rad/s at full stick and 10 rad/s at full stick in turbo. It is installed on the board but not yet in the repository.

3.8 CHASE NODE

`chase_node.py` runs a MobileNet-SSD spatial detection network on the OAK-D-Lite: the 300×300 RGB preview feeds the network on the camera’s VPU, and the stereo depth (aligned to RGB) gives each detection a 3-D position (x, z) in the camera frame, x to the right and z forward. The node keeps the most confident detection whose label is in `targets` and computes a bearing and range,

$$\beta = \text{atan2}(x, z), \quad \rho = \sqrt{x^2 + z^2}, \quad (3.2)$$

falling back to $\beta = (c_x - \frac{1}{2})\text{HFOV}$ from the box centre $c_x \in [0, 1]$ when the box has no valid depth. Both are smoothed with an exponential moving average of weight $a = 0.5$. After at least 3 consecutive detections the command is

$$\omega = \begin{cases} \text{sat}_{\omega_{\max}}(-k_\beta \beta) & |\beta| > \beta_{\text{db}} \\ 0 & \text{otherwise} \end{cases} \quad (3.3)$$

$$v = \text{sat}_{[0, v_{\max}]}(k_\rho(\rho - \rho_f)) \cdot \text{sat}_{[0, 1]}\left(1 - \frac{|\beta|}{\beta_{\text{full}}}\right), \quad \rho > \rho_{\text{stop}}, \quad (3.4)$$

with $k_\beta = 2.5$, $\omega_{\max} = 2$ rad/s, $\beta_{\text{db}} = 5^\circ$, $k_\rho = 0.4$, $v_{\max} = 0.2$ m/s, follow distance $\rho_f = 0.9$ m, $\beta_{\text{full}} = 25^\circ$ and stop distance $\rho_{\text{stop}} = 0.5$ m. The second factor of v slows the robot while it is still turning toward the target. The robot never reverses.

Command arbitration. Both teleop and chase publish `/cmd_vel`, and `bbai_base` applies whichever arrives last. Conflicts are avoided by the buttons: chase only publishes while R2 is held and L1 is not, and teleop only publishes while L1 is held. Chase also stops if `/joy` goes quiet

for 0.5 s or the target is lost for 0.5 s. In the default dry-run mode chase never publishes `/cmd_vel` at all, only `/chase/cmd_vel_preview`.

Chapter 4

DRIVE KINEMATICS AND WHEEL ODOMETRY

4.1 NOTATION

Table 4.1 lists the geometric parameters, with the values in `ros/bbai_base/config/base.yaml`.

Table 4.1: Geometry and drive parameters

Symbol	Meaning	Value	base.yaml key
r	wheel radius	0.0396 m	<code>wheel_radius</code>
b	track (left–right wheel centres)	0.120 m	<code>track_width</code>
L	wheelbase (front–back axles)	0.101 m	—
N	encoder counts per wheel revolution	15	<code>counts_per_rev</code>
Δt	odometry period	1/30 s	<code>rate</code>
$v_{\max}$	wheel speed at duty 1 (lifted)	1.2 m/s	<code>max_wheel_speed</code>
$d_{\max}$	duty limit	0.60	<code>max_duty</code>
$d_{\min}$	duty floor for any nonzero command	0.12	<code>min_duty</code>
T_{cmd}	command timeout	0.5 s	<code>cmd_timeout</code>

The pose in the `odom` frame is $\mathbf{p} = [x \ y \ \psi]^\top$. Subscript k denotes the k -th cycle of the 30 Hz loop, and $\Delta(\cdot)_k$ the change between cycles $k - 1$ and k .

4.2 FROM ENCODER COUNTS TO WHEEL TRAVEL

Let c_j be the raw count of encoder channel $j \in \{1, 2, 3, 4\}$ and $\sigma_j \in \{+1, -1\}$ its sign from Table 2.3. One count is an arc of

$$\delta = \frac{2\pi r}{N} = \frac{2\pi(0.0396)}{15} = 0.01659 \text{ m.} \quad (4.1)$$

Each side averages its two wheels,

$$s_L = \frac{\delta}{2}(\sigma_1 c_1 + \sigma_3 c_3), \quad s_R = \frac{\delta}{2}(\sigma_2 c_2 + \sigma_4 c_4), \quad (4.2)$$

and the travel of the body centre in one cycle is

$$d_k = \frac{1}{2}(\Delta s_{L,k} + \Delta s_{R,k}). \quad (4.3)$$

Averaging both wheels per side halves the effect of a single wheel slipping or being spun by hand: a spurious count on one wheel moves its side’s average by half a count.

Remark 4.1 (Quantization). With $N = 15$ the encoders are coarse. One count is 16.6 mm of wheel travel, so d_k is quantized in steps of $\delta/4 = 4.1$ mm. The velocity published on `/odom`, $v_k = d_k/\Delta t$, is therefore quantized in steps of $\delta/(4\Delta t) = 0.124$ m/s at 30 Hz: at a steady 0.3 m/s it alternates between about 2 and 3 levels from one message to the next. This is the same trade the thesis identifies for its motor encoders (the sample-rate remark in thesis §7.5): raising the loop rate makes the differenced velocity noisier for a fixed encoder resolution. Position, which is a sum of counts, is not affected; only the per-message velocity is. Anything that consumes the twist (the EKF in particular) should either use a longer differencing window or carry a velocity variance of at least $\delta^2/(192 \Delta t^2) \approx 1.3 \times 10^{-3} \text{ m}^2/\text{s}^2$, the variance of a uniform quantization error of width $\delta/(4\Delta t)$.

4.3 SKID-STEER KINEMATICS

For a differential-drive robot with no wheel slip the body speed and yaw rate are

$$v = \frac{1}{2}(v_R + v_L), \quad \omega = \frac{v_R - v_L}{b}. \quad (4.4)$$

A four-wheel skid-steer robot cannot turn without its wheels sliding sideways, because the front and back wheels on each side are fixed parallel at a distance L apart. The instantaneous centres of rotation of the left and right treads move outward, which is captured by an effective track χb with $\chi \geq 1$ [4, 5]:

$$\omega = \frac{v_R - v_L}{\chi b}. \quad (4.5)$$

χ depends on the ratio L/b (here 0.84, which is high), the floor surface, the load and the turn rate itself; values of 1.5 to 2 are typical. An encoder-only heading using (4.4) would overestimate every turn by the factor χ , and that factor would change between carpet and tile. This is why the base node takes its heading from the gyroscope (`use_gyro_heading: true`) and uses the encoders only for distance.

4.4 ODOMETRY INTEGRATION

The heading increment is

$$\Delta\psi_k = \begin{cases} (g_{z,k} - \hat{b}_g)\Delta t_k & \text{gyro heading (default)} \\ \frac{\Delta s_{R,k} - \Delta s_{L,k}}{b} & \text{encoder heading} \end{cases} \quad (4.6)$$

where g_z is the z gyroscope rate and $\hat{b}_g$ the bias estimate (§5.3). While the robot is parked, $\Delta\psi_k = 0$ (§5.3.3). The pose is advanced with the midpoint rule (second-order Runge–Kutta),

$$\begin{aligned}x_k &= x_{k-1} + d_k \cos\left(\psi_{k-1} + \frac{1}{2}\Delta\psi_k\right), \\y_k &= y_{k-1} + d_k \sin\left(\psi_{k-1} + \frac{1}{2}\Delta\psi_k\right), \\ \psi_k &= \text{wrap}(\psi_{k-1} + \Delta\psi_k),\end{aligned}\tag{4.7}$$

with $\text{wrap}(\alpha) = \text{atan2}(\sin \alpha, \cos \alpha) \in (-\pi, \pi]$. For motion on a circular arc the exact update is

$$x_k = x_{k-1} + \frac{d_k}{\Delta\psi_k} \left[\sin(\psi_{k-1} + \Delta\psi_k) - \sin \psi_{k-1} \right],\tag{4.8}$$

and similarly for y . Expanding both in $\Delta\psi_k$ shows that the midpoint rule differs from the exact arc only by a factor $1 - \Delta\psi_k^2/24$ on the step length. At the fastest turn the robot makes, 10 rad/s, $\Delta\psi_k = 0.33$ rad and the error is 0.5% of d_k , and in a pure spin $d_k \approx 0$ anyway, so (4.7) is adequate.

The message also carries the body velocity $v_k = d_k/\Delta t_k$ and $\omega_k = \Delta\psi_k/\Delta t_k$, and the same pose is broadcast as the transform `odom→base_link` with quaternion $\mathbf{q} = [0, 0, \sin \frac{\psi}{2}, \cos \frac{\psi}{2}]$.

4.4.1 Odometry uncertainty

Linearising (4.7) about the current estimate gives the usual dead-reckoning covariance propagation [6],

$$P_k = F_k P_{k-1} F_k^\top + G_k Q_k G_k^\top, \quad F_k = \begin{bmatrix} 1 & 0 & -d_k \sin \bar{\psi} \\ 0 & 1 & d_k \cos \bar{\psi} \\ 0 & 0 & 1 \end{bmatrix}, \quad G_k = \begin{bmatrix} \cos \bar{\psi} & 0 \\ \sin \bar{\psi} & 0 \\ 0 & 1 \end{bmatrix},\tag{4.9}$$

where $\bar{\psi} = \psi_{k-1} + \frac{1}{2}\Delta\psi_k$ and $Q_k = \text{diag}(\sigma_d^2, \sigma_\psi^2)$ is the variance of one step’s distance and heading increments. The position uncertainty grows without bound, and the heading term couples into it through the off-diagonal entries of F_k : a heading error $\delta\psi$ produces a lateral error $d\delta\psi$ after travelling d .

Remark 4.2 (The published covariance is constant). `base_node.py` publishes a fixed diagonal covariance, $(0.01, 0.01, 10^6, 10^6, 10^6, 0.05)$ for $(x, y, z, \phi, \theta, \psi)$ on both the pose and the twist. The 10^6 entries correctly mark the unobserved states. The pose entries, however, claim a 0.1 m standard deviation forever, while (4.9) grows. A filter that fuses the `/odom pose` as an absolute measurement will therefore trust it far too much after a few metres. The safe use is to fuse only the twist, or the pose in `robot_localization`’s `differential` mode. The GPS EKF does the former: it takes only v_x from `/odom` (§7.3).

4.5 FROM VELOCITY COMMAND TO MOTOR DUTY

A `/cmd_vel` message (v, ω) is turned into a desired speed for each side using the differential model (4.4),

$$v_L = v - \frac{1}{2}\omega b, \quad v_R = v + \frac{1}{2}\omega b, \quad (4.10)$$

and each side speed into a PWM duty for both of that side's motors,

$$u = \frac{v_{\text{side}}}{v_{\text{max}}}, \quad D = \text{sat}_{d_{\text{max}}}(\text{sgn}(u)[d_{\text{min}} + (1 - d_{\text{min}})|u|]), \quad (D = 0 \text{ if } |u| < 10^{-3}), \quad (4.11)$$

multiplied by the motor's sign. The offset d_{min} is a deadzone compensator: below about 8% duty the wheels do not turn at all (lifted test), so any nonzero command starts at 12%. This is the inverse of the deadzone nonlinearity the thesis models for its own motors (thesis §7.6.2). The control is open loop: duty sets voltage, and the speed that results depends on load and battery voltage. The encoders are used for odometry but not for speed feedback.

Table 4.2 evaluates (4.10)–(4.11) for the commands teleop sends. The duty limit of 0.6 is reached at $|u| = 0.545$, so the fastest either side can be commanded is about 0.65 m/s.

Table 4.2: Commanded duty for typical commands

Command	v_L (m/s)	v_R (m/s)	D_L	D_R
Forward 0.3 m/s	0.30	0.30	0.34	0.34
Spin 1.5 rad/s (old full stick)	−0.09	0.09	−0.19	0.19
Spin 6 rad/s (new full stick)	−0.36	0.36	−0.38	0.38
Spin 10 rad/s (new turbo)	−0.60	0.60	−0.56	0.56
Forward 0.3 m/s + 1.5 rad/s	0.21	0.39	0.27	0.41

4.6 WHY SKID-STEER TURNING NEEDS SO MUCH POWER

Jason observed on 2 October 2026 that normal-speed turning would not start the robot spinning from rest and barely turned it while moving. A simple friction model explains this [7]. Let the robot weigh W , shared equally by the four wheels, which sit at $\pm L/2$ from the centre along the body and $\pm b/2$ across it. Spinning in place, each wheel must slide sideways, and the floor resists with a lateral friction force $\mu_\ell W/4$ acting at a lever arm of $L/2$. The total resisting moment is

$$M_r = 4 \cdot \mu_\ell \frac{W}{4} \cdot \frac{L}{2} = \frac{\mu_\ell W L}{2}. \quad (4.12)$$

The motors supply it with opposing tractive forces $\pm F$ per side at lever arm $b/2$, a moment of $F b$. Turning starts only when

$$F \geq \frac{\mu_\ell L}{2b} W \approx 0.42 \mu_\ell W. \quad (4.13)$$

Driving straight only has to overcome rolling resistance $f_r W$ with f_r of order 0.02–0.05, whereas lateral sliding friction μ_ℓ is of order 0.3–0.7 on household floors. A spin therefore needs roughly 3 to 15 times the tractive force of driving straight. With the old linear mapping, full stick asked for 1.5 rad/s, which by Table 4.2 is a duty of only 0.19, barely above the 0.12 floor; that is enough to keep a lifted wheel turning but not to make the tyres scrub. This motivated the new teleop turn curve (6 rad/s at full stick). A closed-loop wheel-speed controller using the encoders, or a separate duty floor for turning, would make turn response independent of the floor surface.

Remark 4.3. (4.13) shows the turn force grows with L/b . Moving the wheels farther apart, or using wheels with less lateral grip, would reduce it.

Chapter 5

ATTITUDE AND HEADING

The robot drives on flat floors, so the navigation state needs only the yaw ψ from the IMU. Roll and pitch matter only for tilt-compensating the magnetometer. This chapter derives the gyroscope heading used by the odometry, its error growth, the accelerometer tilt, and the magnetometer compass.

5.1 SENSOR FRAME

`librobotcontrol` returns the gyroscope, accelerometer and magnetometer in one common frame: the AK8963 magnetometer inside the MPU-9250 has its x and y axes swapped and z reversed relative to the gyroscope, and the library undoes that (`mpu.c`, $m_x \leftarrow m'_y$, $m_y \leftarrow m'_x$, $m_z \leftarrow -m'_z$). The static transform `base_link`→`imu_link` has zero rotation, which assumes the cape's IMU axes coincide with the robot's x -forward, y -left, z -up axes. The yaw-rate sign has been confirmed in practice (gyro heading turns the right way in RViz and agrees with the compass over two turns); the x and y axes have not been checked separately.

5.2 SENSOR ERROR MODEL

Following the thesis (thesis §7.5, after Gebre-Egziabher [8] and Flenniken [9]), each inertial channel is modelled as

$$m = m_r + c_r + b_r + w_r, \quad (5.1)$$

where m_r is the true value, c_r a constant bias that changes at every power-up, b_r a walking bias and w_r white noise of variance σ_r^2 . The walking bias is a first-order Gauss–Markov process,

$$\dot{b}_r = -\frac{1}{\tau_r} b_r + w_{b_r}, \quad (5.2)$$

with w_{b_r} white of intensity $2\sigma_{b_r}^2/\tau_r$, so that $\sigma_{b_r}^2$ is its steady-state variance. The thesis characterised an MPU-class gyroscope this way, with Allan variance, and found $\tau_r \approx 1600$ s (thesis Table 7.2); the

same procedure can be reused on this IMU by logging `/imu/data_raw` at rest for an hour.

5.3 GYROSCOPE HEADING

5.3.1 Bias removal at start-up

At start the base node averages the z rate for $T_b = 2$ s with the robot stationary,

$$\hat{b}_g = \frac{1}{n} \sum_{i=1}^n g_{z,i}, \quad (5.3)$$

which estimates $c_r + b_r(0)$ in (5.1). This is the same start-up bias removal used on the TWIP (thesis §4.3). The corrected rate $g_z - \hat{b}_g$ is integrated in (4.6).

5.3.2 Heading error growth

The heading error after time t has three parts. Integrating white noise gives an angle random walk; an error $\delta b = \hat{b}_g - (c_r + b_r(0))$ in the start-up estimate gives a linear ramp; and the walking bias adds a slower term,

$$\delta\psi(t) = \underbrace{\int_0^t w_r ds}_{\sigma=N_g\sqrt{t}} + \underbrace{\delta b t}_{\text{start-up error}} + \int_0^t (b_r(s) - b_r(0)) ds. \quad (5.4)$$

The MPU-9250 datasheet gives a rate-noise density of $N_g = 0.01^\circ/\text{s}/\sqrt{\text{Hz}}$. The random-walk term is therefore $0.01^\circ\sqrt{t}/\text{s}$, under 0.1° after one minute. Averaging over $T_b = 2$ s leaves δb with a standard deviation of about $N_g/\sqrt{T_b} = 0.007^\circ/\text{s}$, which would give about $0.4^\circ/\text{min}$.

The measured drift at rest was $1.6^\circ/\text{min}$ ($0.027^\circ/\text{s}$), four times larger. The noise terms cannot account for it, so most of it is bias that moves after the start-up average: temperature change as the board and motor drivers warm up (the datasheet allows the zero-rate offset to move by up to $\pm 30^\circ/\text{s}$ over the full temperature range) and the walking bias. Over a 10-minute mapping run this is about 16° of heading error if nothing corrects it. Two practical consequences:

- Measuring the bias after the robot has been powered for a few minutes will reduce the error.
- The bias should keep being estimated while the robot runs. The base node now does this whenever the robot is parked (§5.3.3); the magnetometer can also bound the heading (§5.6).

5.3.3 Zero-velocity update

When the robot is parked its true yaw rate is zero, so whatever the gyroscope reads is bias. The base node uses this. Let t_m be the last time any wheel tick changed or a `/cmd_vel` command was

active. If $t - t_m > T_z = 0.5$ s the robot is declared stationary and, each cycle,

$$\hat{b}_{g,k} = \hat{b}_{g,k-1} + \alpha(g_{z,k} - \hat{b}_{g,k-1}), \quad \alpha = \min\left(1, \frac{\Delta t}{\tau_z}\right), \quad \Delta\psi_k = 0, \quad (5.5)$$

with $\tau_z = 20$ s. The first part is a first-order low-pass filter (time constant τ_z) that tracks the walking bias b_r of (5.2); the second freezes the heading. In Kalman filter terms this is a pseudo-measurement $\omega = 0$ with zero variance on the heading and a steady-state gain α on the bias [10]. After the change, heading drift while parked fell from about $1.5^\circ/\text{min}$ to zero (2 October 2026). Drift while driving is still governed by (5.4), but every stop now re-estimates the bias.

Remark 5.1. The stationary test uses the wheels and the command, not the gyroscope. If the robot is picked up and turned by hand, or pushed sideways with the wheels locked, the heading will not follow, and the bias filter will absorb part of the real rotation. Carry the robot level and still, or restart `bbai_base`, when moving it by hand.

5.4 TILT FROM THE ACCELEROMETER

The accelerometer measures specific force, which at rest is gravity in body axes. With roll ϕ and pitch θ , the thesis derives (thesis eq. 4.2, after Kuipers [11])

$$\phi = \text{atan2}(a_y, a_z), \quad \theta = \text{atan2}\left(-a_x, \sqrt{a_y^2 + a_z^2}\right). \quad (5.6)$$

This assumes no acceleration other than gravity, which is close enough for this robot’s low accelerations, and could be filtered with the complementary filter of the thesis (thesis eq. 4.4) if needed. The current software does not use roll and pitch.

5.5 MAGNETOMETER COMPASS

5.5.1 Measurement model

The magnetometer measures Earth’s field $\mathbf{h}$ rotated into the body frame, distorted by nearby material, plus noise [12]:

$$\mathbf{m} = S R^T \mathbf{h} + \mathbf{b}_{\text{hi}} + \mathbf{w}_m. \quad (5.7)$$

R is the body-to-world rotation. The hard-iron offset $\mathbf{b}_{\text{hi}}$ is a constant field carried with the robot: permanent magnets in the motors and encoder rings, and steady currents. The matrix S combines the sensor’s scale factors, axis misalignment and soft-iron effects, where nearby steel bends Earth’s field. On this robot the measured field magnitude was about $150 \mu\text{T}$ against Earth’s $50 \mu\text{T}$, so $\mathbf{b}_{\text{hi}}$ dominates.

5.5.2 Planar calibration

When the robot spins on a level floor, R is a rotation about z alone and the horizontal components (m_x, m_y) trace an ellipse; without soft-iron distortion it is a circle of radius H (the horizontal field) centred on (b_x, b_y) . The calibration routine (`mag_calibrate.py`) commands 5 rad/s until the gyroscope has seen two full turns (or 40 s pass), logging (m_x, m_y) from `/imu/mag`. A lower command stalls on the floor (§4.6).

Circle fit (used). The routine fits a circle by linear least squares (the Kåsa fit [13]): writing the circle as $m_x^2 + m_y^2 + Dm_x + Em_y + F = 0$, each sample gives one linear equation in (D, E, F) , and the normal equations

$$\begin{bmatrix} \sum m_x^2 & \sum m_x m_y & \sum m_x \\ \sum m_x m_y & \sum m_y^2 & \sum m_y \\ \sum m_x & \sum m_y & n \end{bmatrix} \begin{bmatrix} D \\ E \\ F \end{bmatrix} = - \begin{bmatrix} \sum m_x z \\ \sum m_y z \\ \sum z \end{bmatrix}, \quad z = m_x^2 + m_y^2, \quad (5.8)$$

are solved by Cramer’s rule. The hard-iron offset is the centre $(o_x, o_y) = (-D/2, -E/2)$ and the horizontal field is $H = \sqrt{o_x^2 + o_y^2 - F}$. The routine also unwraps the compass heading over the spin and compares its direction with the gyroscope’s to set `mag_sign` $\in \{+1, -1\}$. The result on 1 October 2026 was $(o_x, o_y) = (11.71, 66.61)$ μT , $s = +1$. A circle fit removes the hard iron only; if the residual (printed as “fit error”) grows, one of the estimators below adds soft-iron correction.

Min–max (offset and per-axis scale).

$$b_x = \frac{1}{2}(m_x^{\max} + m_x^{\min}), \quad s_x = \frac{\bar{\rho}}{\frac{1}{2}(m_x^{\max} - m_x^{\min})}, \quad (5.9)$$

and likewise for y , with $\bar{\rho}$ the mean of the two half-ranges. The corrected field is $m'_x = s_x(m_x - b_x)$, $m'_y = s_y(m_y - b_y)$. This handles hard iron and unequal axis gains but not rotation of the ellipse.

Ellipse fit (full planar soft iron). Fit the conic

$$Am_x^2 + Bm_x m_y + Cm_y^2 + Dm_x + Em_y = 1 \quad (5.10)$$

to all samples by linear least squares. The centre $\mathbf{b} = -\frac{1}{2}M^{-1}[D \ E]^T$ with $M = \begin{bmatrix} A & B/2 \\ B/2 & C \end{bmatrix}$ is the hard-iron offset, and the correction $\mathbf{m}' = M^{1/2}(\mathbf{m} - \mathbf{b})$ maps the ellipse back to a circle.

The routine prints the resulting lines for `base.yaml`. The check during calibration was that over the two-turn spin the compass heading advanced 745° while the gyroscope advanced 724° , against 720° actual. The gyroscope’s scale factor is therefore good to about 0.6% and the compass’s residual distortion is about 3.5%.

5.5.3 Heading from the field

On level ground, write the world (ENU) field's horizontal part in terms of the magnetic declination D (positive when magnetic north is east of true north): $\mathbf{h}_{\text{hor}} = H[\sin D, \cos D]^T$ in (east, north). With the robot at ENU yaw ψ , the body-frame components are

$$m'_x = H \sin(\psi + D), \quad m'_y = H \cos(\psi + D), \quad (5.11)$$

so

$$\psi = \text{atan2}(m'_x, m'_y) - D. \quad (5.12)$$

This is exactly what `base_node.py` computes, with $m' = s(m - o)$ from the circle fit: $\psi_m = s \text{atan2}(m_x - o_x, m_y - o_y) - D$. It publishes ψ_m as the orientation of `/imu/data` with variance 0.05 rad^2 ($\sigma \approx 13^\circ$) on yaw and 10^3 on roll and pitch (unknown), and the gyroscope rate with variance $1 \times 10^{-4} \text{ rad}^2/\text{s}^2$. Checking the two easy cases: facing east ($\psi = 0$) with $D = 0$, north is on the robot's left, so $m'_y = H$, $m'_x = 0$ and $\psi = 0$; facing north ($\psi = \pi/2$), north is straight ahead, $m'_x = H$ and $\psi = \pi/2$.

Two conventions are easy to confuse here. A compass heading χ_c is measured clockwise from north, while ROS yaw is counter-clockwise from east:

$$\chi_c = \frac{\pi}{2} - \psi. \quad (5.13)$$

The robot's declination is set to 2.5° east, computed from the NOAA World Magnetic Model [14] for the robot's home location. (The coordinates themselves are deliberately not recorded in this document or the repository.)

5.5.4 Tilt compensation

If the robot is not level, the levelled field is

$$\mathbf{m}_h = R_y(\theta)R_x(\phi)\mathbf{m}', \quad (5.14)$$

using the ZYX convention in which $R = R_z(\psi)R_y(\theta)R_x(\phi)$, and (5.12) is applied to $(m_{h,x}, m_{h,y})$. With ϕ and θ from (5.6). On a level floor $\mathbf{m}_h = \mathbf{m}'$. A 5° tilt with a magnetic inclination of about 60° (typical of the southern United States) produces up to about 9° of heading error, so compensation matters on ramps and lawns but not indoors.

5.5.5 Limits indoors

Indoors, steel in floors, appliances and wiring adds local fields that move the compass by tens of degrees as the robot drives. The calibration above removes only fields carried with the robot. The compass is therefore useful outdoors for initialising the GPS frame (§6.3) but should not be trusted as an absolute heading inside the house.

5.6 FUSING GYRO AND COMPASS

The gyroscope is precise over seconds and drifts over minutes; the compass is noisy and locally biased but does not drift. The thesis used exactly this structure for tilt (thesis eq. 4.4), and the heading version is

$$\psi_{k+1} = \psi_k + (g_{z,k} - \hat{b}_g)\Delta t + (1 - \lambda) \text{wrap}(\psi_{m,k} - \psi_k - (g_{z,k} - \hat{b}_g)\Delta t), \quad (5.15)$$

where ψ_m is (5.12). The wrap is needed because the two headings can differ across the $\pm\pi$ boundary. The filter has a crossover time constant $\tau = \lambda\Delta t/(1 - \lambda)$: with $\Delta t = 1/30$ s, $\lambda = 0.999$ gives $\tau \approx 33$ s. Higgins [15] shows this is the steady-state Kalman filter for a random-walk heading with white measurement noise, which is the form the EKF of Chapter 7 implements with time-varying gains. (5.15) is not currently running. The compass feeds only the GPS EKF on the laptop; the odometry heading that gmapping uses is gyroscope only, which is deliberate given the indoor limits above.

Chapter 6

GPS MEASUREMENT MODEL

6.1 NMEA TO NAVSATFIX

`gps_node.py` reads `/dev/ttyS2` one line at a time, drops any sentence whose checksum fails (the XOR of every character between \$ and *, compared with the two hex digits after *), and handles two sentence types.

GGA (position). Latitude and longitude arrive as `ddmm.mmmm` and `dddmm.mmmm`. With the integer degrees d and minutes m ,

$$\varphi = \pm \left(d + \frac{m}{60} \right), \quad (6.1)$$

negative for S and W. The altitude published is the ellipsoidal height $h = H_{\text{MSL}} + N$, where H_{MSL} is the antenna altitude above mean sea level (field 9) and N the geoid separation (field 11). A quality field of 0 publishes `STATUS_NO_FIX` with NaN coordinates; quality 2 (differential/SBAS) publishes `STATUS_SBAS_FIX`.

Position covariance. The receiver does not report an error estimate, only the horizontal dilution of precision (HDOP). The node uses the standard approximation [16]

$$\sigma_h = \text{HDOP} \cdot \sigma_{\text{UERE}}, \quad \Sigma_{\text{ENU}} = \text{diag}(\sigma_h^2, \sigma_h^2, 4\sigma_h^2), \quad (6.2)$$

with a user-equivalent range error $\sigma_{\text{UERE}} = 5$ m and the vertical taken as twice the horizontal. With a typical open-sky HDOP of 1 this is a 5 m standard deviation, and the EKF automatically trusts a poor fix less.

RMC (velocity). Speed over ground s (knots, times 0.514 444 m/s) and course over ground c (degrees clockwise from true north) become an ENU velocity,

$$v_E = s \sin c, \quad v_N = s \cos c, \quad (6.3)$$

published on `/gps/vel` only while there is a fix. The EKF does not currently use it.

6.2 GEODETIC TO LOCAL COORDINATES

The EKF works in metres, so latitude φ , longitude λ and height h must be turned into a local Cartesian frame. On the WGS-84 ellipsoid (semi-major axis $a = 6\,378\,137$ m, eccentricity $e^2 = 6.694\,379\,99 \times 10^{-3}$) the Earth-centred, Earth-fixed position is

$$\begin{aligned} X &= (R_N + h) \cos \varphi \cos \lambda, \\ Y &= (R_N + h) \cos \varphi \sin \lambda, \\ Z &= (R_N(1 - e^2) + h) \sin \varphi, \end{aligned} \quad R_N = \frac{a}{\sqrt{1 - e^2 \sin^2 \varphi}}, \quad (6.4)$$

and the ENU offset from a reference point (φ_0, λ_0) with ECEF position $\mathbf{r}_0$ is

$$\begin{bmatrix} e \\ n \\ u \end{bmatrix} = \begin{bmatrix} -\sin \lambda_0 & \cos \lambda_0 & 0 \\ -\sin \varphi_0 \cos \lambda_0 & -\sin \varphi_0 \sin \lambda_0 & \cos \varphi_0 \\ \cos \varphi_0 \cos \lambda_0 & \cos \varphi_0 \sin \lambda_0 & \sin \varphi_0 \end{bmatrix} (\mathbf{r} - \mathbf{r}_0). \quad (6.5)$$

Over the few hundred metres a robot this size covers, (6.5) reduces to the flat-Earth approximation

$$e \approx (R_N + h) \cos \varphi_0 \Delta \lambda, \quad n \approx (R_M + h) \Delta \varphi, \quad R_M = \frac{a(1 - e^2)}{(1 - e^2 \sin^2 \varphi_0)^{3/2}}, \quad (6.6)$$

with angles in radians. At mid-latitudes one degree of latitude is about 111 km, so the EM-506's 5 m error is about 4.5×10^{-5} degrees.

6.3 THE NAVSAT TRANSFORM

robot_localization's `navsat_transform_node` [17] uses the Universal Transverse Mercator (UTM) projection instead of (6.5): each fix becomes a UTM easting and northing $\mathbf{u}$, which is locally an ENU frame rotated by the small grid convergence angle (under 2° inside a zone, ignored here). The node builds a fixed transform from UTM to the EKF's world frame (`gps_map`) once, at the *datum*:

1. With `wait_for_datum: false`, the datum is the first fix that arrives (after a 2 s delay), together with the robot's pose in the EKF output $\mathbf{p}_0 = (x_0, y_0, \psi_0)$ and the IMU heading $\psi_{\text{imu},0}$ at that moment.
2. The robot's heading in UTM is $\psi_{\text{utm},0} = \psi_{\text{imu},0} + D_{\text{nst}} + \psi_{\text{off}}$. On this robot the declination is already applied on the board and `/imu/data` is ENU yaw, so `magnetic_declination_radians` $D_{\text{nst}} = 0$ and `yaw_offset` $\psi_{\text{off}} = 0$.
3. The transform maps a UTM point to the world frame by undoing the robot's UTM pose at

the datum and applying its world pose:

$$\mathbf{x}_w = R(\psi_0) R(\psi_{\text{utm},0})^\top (\mathbf{u} - \mathbf{u}_0) + \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}, \quad (6.7)$$

where $R(\cdot)$ is the planar rotation matrix and $\mathbf{u}_0$ the datum's UTM position (offset by the antenna lever arm, here zero).

Every later fix is pushed through (6.7) and published on `/odometry/gps` with the covariance (6.2) rotated by the same angle. That message is what the EKF fuses. The node also runs the inverse transform on the EKF output and publishes `/gps/filtered`, the fused position as latitude and longitude. `zero_altitude` is set, so height is ignored.

Remark 6.1 (The datum heading matters). Equation (6.7) rotates every GPS fix by the compass heading taken at a single instant. A heading error $\delta\psi$ at the datum rotates the whole GPS track about the starting point, so a fix ℓ metres away lands $\ell\delta\psi$ off to the side: 10° of compass error puts a point 20 m away 3.5 m off. The datum should therefore be taken outdoors, away from steel, with the robot still.

Chapter 7

SENSOR FUSION: THE GPS EKF

The global estimate combines wheel odometry, the gyroscope, the compass and GPS in an extended Kalman filter (EKF) from the `robot_localization` package [17], running on the laptop as node `ekf_gps` (`laptop/bbai_slam/gps_fusion.launch` and `gps_ekf.yaml`). This chapter states the filter’s model, the measurement model of each sensor, and how the result relates to the SLAM map.

7.1 THE DISCRETE EKF

The thesis gives the linear discrete Kalman filter in predictor–corrector form (thesis §7.7, after Kalman [18] and Simon [19]). The EKF is the same recursion with the state transition f and measurement h nonlinear and their Jacobians used for the covariance:

$$\hat{x}_k^- = f(\hat{x}_{k-1}^+, \Delta t_k), \quad P_k^- = F_k P_{k-1}^+ F_k^\top + Q_k \Delta t_k, \quad (7.1)$$

$$\nu_k = z_k - H_k \hat{x}_k^-, \quad S_k = H_k P_k^- H_k^\top + R_k, \quad (7.2)$$

$$K_k = P_k^- H_k^\top S_k^{-1}, \quad \hat{x}_k^+ = \hat{x}_k^- + K_k \nu_k, \quad (7.3)$$

$$P_k^+ = (I - K_k H_k) P_k^- (I - K_k H_k)^\top + K_k R_k K_k^\top, \quad (7.4)$$

with $F_k = \partial f / \partial \mathbf{x}$ at $\hat{x}_{k-1}^+$. The covariance update (7.4) is the Joseph form, the same one the thesis uses, which keeps P symmetric and positive definite under round-off. Angular components of the innovation are wrapped to $(-\pi, \pi]$. `robot_localization` runs the prediction at a fixed 20 Hz (**frequency**) and applies each sensor message as it arrives, after predicting forward to the message’s time stamp. Measurements may update any subset of the state; H_k is then the rows of the identity that select the components marked **true** in that sensor’s configuration vector.

7.2 STATE AND PROCESS MODEL

`robot_localization` always carries a 15-element state,

$$\mathbf{x} = [x \ y \ z \ \phi \ \theta \ \psi \ \dot{x}_b \ \dot{y}_b \ \dot{z}_b \ \dot{\phi} \ \dot{\theta} \ \dot{\psi} \ \ddot{x}_b \ \ddot{y}_b \ \ddot{z}_b]^\top, \quad (7.5)$$

pose in the world frame and velocities and accelerations in the body frame. With `two_d_mode: true`, z , ϕ , θ and their derivatives are fixed at zero, leaving the planar state $[x, y, \psi, v_x, v_y, \omega, a_x, a_y]$. The process model is the “omnidirectional” constant-acceleration model:

$$\begin{aligned} x_k &= x_{k-1} + (v_x \cos \psi - v_y \sin \psi) \Delta t + \frac{1}{2} (a_x \cos \psi - a_y \sin \psi) \Delta t^2, \\ y_k &= y_{k-1} + (v_x \sin \psi + v_y \cos \psi) \Delta t + \frac{1}{2} (a_x \sin \psi + a_y \cos \psi) \Delta t^2, \\ \psi_k &= \psi_{k-1} + \omega \Delta t, \\ v_{x,k} &= v_{x,k-1} + a_x \Delta t, \quad v_{y,k} = v_{y,k-1} + a_y \Delta t, \quad \omega_k = \omega_{k-1}, \end{aligned} \tag{7.6}$$

with accelerations held constant between updates. Its Jacobian has the familiar dead-reckoning entries $\partial x / \partial \psi = -(v_x \sin \psi + v_y \cos \psi) \Delta t + \dots$ and $\partial y / \partial \psi = (v_x \cos \psi - v_y \sin \psi) \Delta t + \dots$, the same coupling as (4.9). The process noise Q is the package default (a diagonal of small values, 0.05 on position and 0.06 on yaw per second); it has not been tuned for this robot.

7.3 MEASUREMENTS

Table 7.1 lists exactly what is fused. Each input contributes only the components marked, with the covariance its publisher reports.

Table 7.1: EKF inputs (`gps_ekf.yaml`)

Input	Topic	Fused	Variance
odom0	/odom	v_x	$0.01 \text{ m}^2/\text{s}^2$ ($\sigma = 0.1 \text{ m/s}$)
imu0	/imu/data	ψ, ω	ψ : 0.05 rad^2 ($\sigma \approx 13^\circ$); ω : $1 \times 10^{-4} \text{ rad}^2/\text{s}^2$
odom1	/odometry/gps	x, y	$(\text{HDOP} \cdot 5 \text{ m})^2$

Written out, the measurement functions are linear selections,

$$h_{\text{odom}}(\mathbf{x}) = v_x, \quad h_{\text{imu}}(\mathbf{x}) = \begin{bmatrix} \psi \\ \omega \end{bmatrix}, \quad h_{\text{gps}}(\mathbf{x}) = \begin{bmatrix} x \\ y \end{bmatrix}, \tag{7.7}$$

so all of the nonlinearity is in the process model.

Design choices.

- *Odometry contributes speed only.* The odometry pose and its yaw are not fused. The pose covariance from `bbai_base` is constant rather than growing (§4.4), so fusing it would over-trust it; and odometry yaw is the integrated gyroscope, which would double-count the gyroscope rate already fused from `imu0`. v_y is not fused either; the model has no constraint that keeps it at zero, so it is held small only by the process noise. Fusing $v_y = 0$ (a nonholonomic pseudo-measurement) would tighten the filter.
- *Heading is absolute from the compass, rate from the gyroscope.* The filter is then exactly the

Kalman form of the complementary filter (5.15): the gyroscope carries the heading between compass updates, and the compass bounds its drift. With variances of 1×10^{-4} on ω and 0.05 on ψ the compass corrects the heading slowly.

- *The gyroscope bias is not a state.* `robot_localization` has no bias states, so a constant rate error appears as a heading ramp that the compass pulls back. The bias estimate comes from the base node’s zero-velocity update instead (§5.3.3).
- *No outlier gating.* The Mahalanobis thresholds (`*_rejection_threshold`) are left at their default of infinity, so a GPS multipath jump is accepted at its stated covariance. Setting `odom1_pose_rejection_threshold` to about 3–5 (rejecting innovations with $\nu^T S^{-1} \nu$ above that many standard deviations squared) would guard against it.

7.4 OBSERVABILITY AND BEHAVIOUR WITHOUT A FIX

With a GPS fix the filter observes x , y and ψ directly and the covariance stays bounded. Without one (always, indoors), x and y are not observed: the filter integrates the odometry speed along the compass-and-gyro heading, and P_{xx} and P_{yy} grow as in (4.9). That is the “coasting” behaviour: `/odometry/global` is then a dead-reckoning estimate whose heading is anchored to the compass rather than to the gyro’s start-up direction. When fixes return, the first few updates pull the estimate onto the GPS track with a jump proportional to the drift accumulated.

7.5 RELATION TO THE SLAM MAP

The EKF publishes no transform (`publish_tf: false`). Its output lives in its own world frame `gps_map`, while `gmapping` owns `map→odom`. The two estimates run side by side:

	SLAM (gmapping)	GPS EKF
World frame	<code>map</code>	<code>gps_map</code>
Absolute reference	the lidar map	GPS and the compass
Works indoors	yes	coasts only
Publishes TF	<code>map→odom</code>	none
Consumers	RViz, <code>map_saver</code>	topic only

Both start from the same odometry, so for short runs they agree up to a fixed rotation and translation. This keeps the two from fighting, but nothing yet relates `map` to `gps_map` or to latitude and longitude.

Aligning the frames. If the robot drives a path outdoors where both are valid, the rigid transform between them follows from the paired positions $(\mathbf{a}_i, \mathbf{g}_i)$ in `map` and `gps_map` by least squares [20]:

$$\theta^* = \text{atan2}\left(\sum_i \bar{\mathbf{g}}_i \times \bar{\mathbf{a}}_i, \sum_i \bar{\mathbf{g}}_i \cdot \bar{\mathbf{a}}_i\right), \quad \mathbf{t}^* = \bar{\mathbf{a}} - R(\theta^*)\bar{\mathbf{g}}, \quad (7.8)$$

where $\bar{\mathbf{a}}, \bar{\mathbf{g}}$ are the means and $\bar{\mathbf{a}}_i, \bar{\mathbf{g}}_i$ the centred points ($\times$ is the planar cross product $g_x a_y - g_y a_x$). Publishing $(\theta^*, \mathbf{t}^*)$ as a static transform `map`→`gps_map` would put GPS fixes on the lidar map. A fuller design, for when the robot navigates on a saved map, replaces gmapping with a localiser (AMCL or slam_toolbox in localization mode) and a second, map-frame EKF that owns `map`→`odom` and fuses both the localiser’s pose and GPS [17].

Chapter 8

MAPPING: LIDAR SLAM

The map is built on the laptop from the board’s `/scan` and the `odom→base_link` transform. Two SLAM packages are set up: `gmapping`, a Rao–Blackwellized particle filter that uses odometry and is the default, and `hector_mapping`, a scan matcher that can run on the lidar alone.

8.1 OCCUPANCY GRID

Both build a 2-D occupancy grid with 5 cm cells. Each cell m_i holds the log-odds of being occupied,

$$\ell_i = \log \frac{p(m_i)}{1 - p(m_i)}, \quad (8.1)$$

and every scan updates the cells along each beam by the inverse sensor model [6]:

$$\ell_{i,t} = \ell_{i,t-1} + \log \frac{p(m_i \mid z_t, x_t)}{1 - p(m_i \mid z_t, x_t)} - \ell_0. \quad (8.2)$$

Cells a beam passes through become more likely free, and the cell where it ends more likely occupied. Because the update is additive, a scan registered at the wrong pose does not erase the old walls; it adds a second, rotated or shifted copy. That is why a pose error shows up as a “jumbled” map rather than a moved one.

8.2 GMAPPING

8.2.1 Rao–Blackwellized particle filter

SLAM estimates the joint posterior of the robot’s path $x_{1:t}$ and the map m , given scans $z_{1:t}$ and odometry $u_{1:t-1}$. `gmapping` [21] factors it as

$$p(x_{1:t}, m \mid z_{1:t}, u_{1:t-1}) = p(m \mid x_{1:t}, z_{1:t}) p(x_{1:t} \mid z_{1:t}, u_{1:t-1}). \quad (8.3)$$

Given a path, the map follows by (8.2), so only the path needs a particle filter; each of the M particles carries one path hypothesis and its own map [22, 23]. Per accepted scan, each particle i :

1. **Predicts** with the odometry motion model: the change in `odom→base_link` since the last update, split into a translation Δ_ρ and a rotation Δ_ψ , is perturbed by Gaussian noise with standard deviations

$$\sigma_\rho = s_{rr} |\Delta_\rho| + s_{rt} |\Delta_\psi|, \quad \sigma_\psi = s_{tr} |\Delta_\rho| + s_{tt} |\Delta_\psi|, \quad (8.4)$$

(parameters `srr`, `srt`, `str`, `stt`).

2. **Scan-matches** the new scan against its own map, starting from the predicted pose x' , by hill-climbing in steps of `lstep` (m) and `astep` (rad) to maximise a likelihood-field score

$$s(x) = \sum_j \exp\left(-\frac{d_j(x)^2}{\sigma}\right), \quad (8.5)$$

where $d_j(x)$ is the distance from the endpoint of beam j (projected from pose x) to the nearest occupied cell, and σ is the parameter `sigma`. Only beams shorter than `maxUrange` are used.

3. **Samples** its new pose from a Gaussian fitted around the best match $\hat{x}^*$ (the “improved proposal”), which combines the scan match with (8.4). If the best score is below `minimumScore`, the match is considered unreliable and the particle falls back to the odometry prediction x' .
4. **Reweights**: $w_t^{(i)} \propto w_{t-1}^{(i)} \eta^{(i)}$, where $\eta^{(i)} = \int p(z_t | m^{(i)}, x) p(x | x_{t-1}^{(i)}, u_{t-1}) dx$ is approximated from the same samples.

When the effective sample size

$$N_{\text{eff}} = \frac{1}{\sum_i (\tilde{w}^{(i)})^2} \quad (8.6)$$

of the normalised weights $\tilde{w}$ falls below `resampleThreshold` · M , particles are resampled. The best particle’s map is published on `/map` every `map_update_interval` seconds, and its pose sets the transform `map→odom` such that

$$T_{\text{map} \leftarrow \text{base_link}} = T_{\text{map} \leftarrow \text{odom}} T_{\text{odom} \leftarrow \text{base_link}}. \quad (8.7)$$

That composition is how the SLAM pose and the odometry pose integrate: the odometry provides a smooth, 30 Hz pose that drifts, and SLAM publishes the slowly changing correction that cancels the drift.

8.2.2 When scans are processed

A scan is processed only if the robot has moved at least `linearUpdate` metres or turned `angularUpdate` radians according to odometry since the last processed scan, or, if `temporalUpdate` is positive, if that many seconds have passed.

8.2.3 Parameters

Table 8.1: gmapping parameters (`laptop/bbai_slam/gmapping_bbai.launch`)

Parameter	1 Oct	2 Oct	Effect
particles M	50	40	path hypotheses
temporalUpdate	2.0 s	off (−1)	process scans while parked
linearUpdate / angularUpdate	0.10 / 0.10	0.10 / 0.10	motion between scans (m / rad)
minimumScore	0	100	reject weak scan matches
srr, srt, str, stt	0.1, 0.2, 0.1, 0.2 [†]	0.05 each	odometry noise (8.4)
sigma, lstep, astep	0.05 [†]	0.05	scan-match kernel and steps
maxUrange / maxRange	8 / 12 m	6 / 12 m	usable / maximum beam length
map_update_interval	2.0 s	1.0 s	map publishing
resampleThreshold	0.5 [†]	0.5	resample when $N_{\text{eff}} < M/2$
delta	0.05 m	0.05 m	cell size
transform_publish_period	0.05 s [†]	0.05 s	map→odom rate

[†] package default, not set in the launch file

In flux. The 2 October values are on branch `slam-estimator-updates` and were being tested when this was written.

8.3 HECTOR MAPPING

`hector_mapping` [24] has no particles and no odometry model. It aligns each scan directly to the map built so far. With the scan’s beam endpoints $\mathbf{s}_j$ in the robot frame and the pose $\boldsymbol{\xi} = (p_x, p_y, \psi)$, the endpoints in the map are

$$\mathbf{S}_j(\boldsymbol{\xi}) = R(\psi)\mathbf{s}_j + \begin{bmatrix} p_x \\ p_y \end{bmatrix}, \quad (8.8)$$

and the pose minimises

$$\boldsymbol{\xi}^* = \arg \min_{\boldsymbol{\xi}} \sum_j [1 - M(\mathbf{S}_j(\boldsymbol{\xi}))]^2, \quad (8.9)$$

where $M(\cdot) \in [0, 1]$ is the occupancy value bilinearly interpolated between grid cells. One Gauss–Newton step from the current estimate is

$$\Delta \boldsymbol{\xi} = H^{-1} \sum_j \left[\nabla M(\mathbf{S}_j) \frac{\partial \mathbf{S}_j}{\partial \boldsymbol{\xi}} \right]^\top [1 - M(\mathbf{S}_j)], \quad H = \sum_j \left[\nabla M \frac{\partial \mathbf{S}_j}{\partial \boldsymbol{\xi}} \right]^\top \left[\nabla M \frac{\partial \mathbf{S}_j}{\partial \boldsymbol{\xi}} \right], \quad (8.10)$$

with

$$\frac{\partial \mathbf{S}_j}{\partial \boldsymbol{\xi}} = \begin{bmatrix} 1 & 0 & -\sin \psi s_{j,x} - \cos \psi s_{j,y} \\ 0 & 1 & \cos \psi s_{j,x} - \sin \psi s_{j,y} \end{bmatrix}. \quad (8.11)$$

The match is run coarse-to-fine on a pyramid of maps (2 levels here) to widen its basin of convergence. Hector converges only if the true pose is within about one coarse cell and a few degrees of the previous estimate, so it fails when the robot turns faster than the scan rate can follow (§8.4). On this robot it is the fallback for mapping without odometry (`lidar_only_bbai.launch`), for example

when carrying the robot by hand.

8.4 SCAN RATE AND ROTATION

The A1 delivers about 7.2 scans per second, so between scans, and during the 0.14 s it takes to sweep one scan, the robot turns by

$$\Delta\psi_{\text{scan}} = \frac{\omega}{f_{\text{scan}}}. \quad (8.12)$$

Turn rate	0.5 rad/s	1.5 rad/s	6 rad/s
Rotation per scan	4°	12°	48°

Neither SLAM package corrects the distortion within a scan (each beam is treated as if taken at the scan’s time stamp), so a scan taken while turning at 1.5 rad/s is itself bent by up to 12° from its first beam to its last. gmapping’s `angularUpdate` of 0.1 rad (5.7°) also means that at 1.5 rad/s nearly every scan is processed while bent. Mapping is most reliable with turns below about 0.5 rad/s. The new teleop curve keeps half stick at 1.5 rad/s; turning gently (well under half stick) while mapping is better still.

8.5 DIAGNOSING THE SLAM DRIFT OF 1 OCTOBER

Jason observed that gmapping made a good first map, lost track once the robot moved, and, when left stationary, slowly rotated the map into a jumble. Each symptom has a distinct likely cause.

Map rotating while parked. With `temporalUpdate` at 2 s, gmapping processed a scan every two seconds even when the robot did not move. The gyroscope heading drifted about 1.6°/min while parked (§5.3), so each new scan arrived with a slightly rotated odometry pose. The scan matcher absorbs a small rotation, but with only about 360 beams per scan and a cell size of 5 cm, its correction is imperfect, and the residual is written into the map by (8.2). Repeated every two seconds this smears the walls into a rotating blur. Two changes address it (branch `slam-estimator-updates`): `temporalUpdate` = −1, so a parked robot adds no scans, and the zero-velocity update (§5.3.3), which removes the heading drift while parked. Parked drift measured afterwards was zero.

Losing track while driving. Several causes can each produce this, and they are best ruled out in order:

1. **Lidar mounting yaw.** The static transform `base_link`→`laser` has zero yaw. `rplidar_ros` reports angle zero along the A1’s own zero direction, which is marked on its housing and is often mounted facing backward. If the transform’s yaw is off by 180°, a spin in place still looks consistent (the scan rotates in the same direction either way), but driving forward makes the walls appear to approach from behind: odometry and scan disagree on every metre travelled and the map breaks as soon as the robot drives. This matches the symptom closely. *Test:* in

RViz with fixed frame `odom`, drive slowly straight toward a wall. The wall should stay still; if it moves, the yaw is wrong. (This check is open and waiting on a drive test.)

2. **Turning too fast.** See §8.4.
3. **Weak scan matches.** At the A1's range and with about 360 beams, narrow corridors and open rooms give the matcher little to hold on to. `minimumScore` now rejects weak matches in favour of odometry, which is reliable over short distances after the calibration (0.914 m measured for 0.914 m). `maxUrange` was reduced to 6 m because the A1's error grows with range.
4. **Odometry noise model.** With `srr-stt` at their defaults (0.1–0.2), particles spread far more than the calibrated odometry warrants, and with only 40–50 particles the filter can lock onto a wrong hypothesis. They are now 0.05.
5. **Clocks.** Scans and transforms are stamped by the board's clock, which has no battery-backed time source and is not synchronised to the laptop. `gmapping` looks up odometry at each scan's stamp, which both come from the board, so this is not expected to cause the drift, but running `chrony` on both machines removes a class of TF extrapolation warnings.

In flux. The lidar thread is working on these. Its results, and any change to the lidar's static transform, belong in `robot.launch` and this section.

Chapter 9

OPERATING MODES

The software is arranged so that each sensor can be left out and the rest still works. This chapter lists the modes that exist today, what each one estimates, and how to start it.

9.1 LOCALIZATION AND MAPPING MODES

Table 9.1: Localization and mapping modes

Mode	Sensors	Start	Position estimate
A. gmapping SLAM (default)	lidar, encoders, gyro	<code>robot.launch;</code> <code>start_slam.sh</code>	map: odometry corrected by scan matching; drift bounded while the map is consistent.
B. hector SLAM with odometry TF	lidar (odometry only as TF)	<code>start_slam.sh</code> <code>hector</code>	map: scan matching only; fails under fast turns.
C. hector, lidar only	lidar	<code>robot.launch</code> , then <code>start_slam.sh</code> <code>lidar_only</code> (or the A1 alone)	As B, with no wheel odometry at all; for carrying the robot.
D. Dead reckoning	encoders, gyro	<code>robot.launch</code> <code>lidar:=false</code>	odom only: distance from encoders, heading from gyro with ZUPT; unbounded drift (§4.4).
E. Encoder heading	encoders	<code>use_gyro_heading:</code> <code>false</code> in <code>base.yaml</code>	odom with heading from (4.4); turns overestimated by χ (§4.3). Diagnostic only.
F. GPS fusion (parallel to A–D)	GPS, compass, gyro, encoders	<code>gps_fusion.launch</code> on the laptop	gps_map: bounded by GPS when there is a fix; coasts on odometry with compass heading otherwise.

Mode F runs alongside any of A–D because it publishes no transform. In practice: indoors, run A and ignore F; outdoors, run F, and A as well if there are walls or obstacles within the lidar’s range.

9.2 WHAT HAPPENS WHEN A SENSOR IS MISSING

Table 9.2: Degradation by sensor

Missing	Effect
GPS fix	Normal indoors. <code>/gps/fix</code> reports <code>NO_FIX</code> ; the EKF coasts. If the first fix comes later, <code>navsat_transform</code> takes its datum then.
Compass	<code>/imu/data</code> not published (<code>mag_heading: false</code> or no field); the EKF has no absolute heading and <code>navsat_transform</code> cannot set a datum, so mode F does not work. Odometry and SLAM are unaffected.
Gyroscope (IMU)	<code>bbai_base</code> cannot start (the IMU is initialised unconditionally). Mode E would be the fallback but needs a code change to skip the IMU.
Encoders	Odometry reports no motion. Mode C still maps. ZUPT would treat the robot as parked whenever no command is active.
One encoder	Its side averages the remaining wheel with a stuck one, so that side reads half its travel; fix the channel list in <code>base.yaml</code> .
Lidar	Modes A–C unavailable; <code>robot.launch lidar:=false</code> gives D.
Motor 1 PRU firmware	<code>/dev/rpmsg_pru32</code> does not appear (the base node logs a warning), <code>rc_motor_init</code> fails, and the motors cannot be driven.
Laptop	Everything on the board runs; no SLAM, no GPS fusion, no RViz.
Bluetooth controller	Motors receive no commands; the 0.5 s timeout keeps them stopped.

9.3 DRIVING MODES

Manual (teleop). Hold L1; left stick drives, right stick turns, R1 adds turbo. Releasing L1 stops the robot.

Chase, dry run. `roslaunch bbai_chase chase.launch`. Holding R2 publishes only the preview topic; nothing moves.

Chase, armed. `chase.launch dry_run:=false`. Holding R2 (with L1 released) lets the chase node drive toward the target. Waiting on separate camera power (§2.12).

Calibration. `roslaunch bbai_base mag_calibrate.py` drives `/cmd_vel` itself to spin two turns. `motor_map_test.py` drives each motor in turn and must be run with `robot.launch` stopped and the wheels lifted.

Chapter 10

KNOWN ISSUES AND NEXT STEPS

10.1 OPEN ISSUES

Board resets under full load. The cape's 5 V rail drops out with the camera, motors, lidar and Bluetooth all running (§2.12). Waiting on a USB splitter and battery bank to power the camera separately.

SLAM loses track while driving. Analysis in §8.5. The parked drift is fixed on branch `slam-estimator-updates`; the lidar mounting yaw is the leading remaining suspect and needs a drive test.

Skid-steer turning. Low turn commands cannot overcome scrubbing friction (§4.6). The teleop curve is a workaround; closed-loop wheel-speed control would fix it properly.

Bluetooth input stalls. Controller reports pause for 100–500 ms in bursts. ROS itself was measured clean (no `/cmd_vel` gap over 58 ms). Parked as acceptable for now.

Placeholder static transforms. The lidar, IMU and GPS offsets in `robot.launch` have not been measured.

Constant odometry covariance. `/odom` reports a fixed pose covariance (§4.4). Harmless for the current EKF, which fuses only v_x , but wrong for any consumer of the pose.

Repository behind the robot. The compass, ZUPT, GPS fusion and gmapping changes are on `slam-estimator-updates`, not `main`. The teleop turn curve is only on the board.

Pause and Mode buttons. The cape's buttons still do not work.

10.2 SUGGESTED IMPROVEMENTS

In rough order of value for the current goals:

1. Verify and set the lidar's yaw in `base_link→laser` (§8.5).
2. Map with gentle turns, under about 0.5 rad/s (§8.4).

3. Close the speed loop: a PI controller per side on encoder speed, with the existing $d_{\min}$ as feed-forward, would make turning independent of the floor and give the chase controller consistent response.
4. Log `/imu/data_raw` at rest for an hour and fit the error model (5.1)–(5.2) with Allan variance, as in the thesis (thesis §7.5). That gives principled process noise for the EKF and a better ZUPT time constant.
5. Publish a growing pose covariance from (4.9) instead of the constant one.
6. Fuse $v_y = 0$ in the EKF and set a Mahalanobis rejection threshold on GPS (§7.3).
7. Consider `slam_toolbox`, which handles loop closure and lifelong mapping better than `gmapping` and can switch to localisation on a saved map.
8. Align `map` and `gps_map` with (7.8) once there is outdoor data.

Appendix A

REFERENCE

A.1 REPOSITORY MAP

Table A.1: Where things are in `jstumfoll2/bbai-robot`

Path	Contents
<code>devicetree/</code>	Robotics Cape device tree for the BB-AI, patch and full sources
<code>librobotcontrol/</code>	Submodule: the <code>v1.1_AIfixes</code> fork
<code>pru/</code>	Notes on the PRU2_0 <code>gpio_rpmsg</code> firmware
<code>system/</code>	<code>uEnv.txt</code> , <code>btsdio</code> blacklist, encoder tick logger
<code>gps/</code>	<code>libgps</code> patch for <code>/dev/ttyS2</code> at 4800 baud
<code>ros/bbai_base/</code>	Base node, GPS node, compass calibration, <code>base.yaml</code> , <code>robot.launch</code>
<code>ros/bbai_teleop/</code>	PS4 teleop node, Bluetooth bring-up and pairing scripts, <code>systemd</code> unit
<code>ros/bbai_chase/</code>	Chase-the-dog node
<code>laptop/bbai_slam/</code>	<code>gmapping</code> , <code>hector</code> and GPS fusion launch files, EKF config, <code>RViz</code> , scripts
<code>camera/</code>	OAK-D-Lite board scripts, PC calibration tools, calibration backup
<code>docs/</code>	Goals, handoff, calibration guide, command cheat sheet, this document

The day-to-day commands for the board, laptop, PC and controller are in `docs/COMMANDS.md`.

A.2 BASE NODE PARAMETERS

A.3 SYMBOLS

A.4 BUILDING THIS DOCUMENT

```
cd docs/robot-system
make # pdflatex + bibtex; or: latexmk -pdf main.tex
```

Table A.2: base.yaml (branch slam-estimator-updates)

Key	Value	Meaning
left_encoders, right_encoders	[1, 3], [2, 4]	encoder channels per side
encoder_signs	{1:1, 2:-1, 3:1, 4:-1}	forward counts up
counts_per_rev	15.0	from the 3 ft push
wheel_radius	0.0396	m
track_width	0.120	m; used only for encoder heading and duty split
use_gyro_heading	true	heading from the gyroscope
publish_tf	true	publish odom→base_link
rate	30.0	Hz
left_motors, right_motors	[1, 2], [3, 4]	motor channels per side
motor_signs	all 1	positive duty drives forward
max_wheel_speed	1.2	m/s at duty 1 (lifted)
max_duty / min_duty	0.6 / 0.12	duty cap and deadzone floor
cmd_timeout	0.5	s
zupt_delay / zupt_bias_tau	0.5 / 20.0	s; defaults in code
mag_heading	true	publish /imu/data
mag_offset_x / _y	11.71 / 66.61	μ T, hard-iron offset
mag_sign	1	compass direction
declination_deg	2.5	east positive

Table A.3: Main symbols

x, y, ψ	planar pose; ψ is ENU yaw, counter-clockwise from east
v, ω	body speed and yaw rate
r, b, L	wheel radius, track, wheelbase
χ	skid-steer effective-track factor
δ	wheel travel per encoder count
$g_z, \hat{b}_g$	gyroscope z rate and its bias estimate
$\mathbf{m}, \mathbf{o}$	magnetometer reading and hard-iron offset
D	magnetic declination, east positive
χ_c	compass heading, clockwise from north ($= \pi/2 - \psi$)
φ, λ, h	latitude, longitude, ellipsoidal height
P, Q, R, K, H	EKF covariance, process noise, measurement noise, gain, measurement matrix
ℓ_i	occupancy log-odds of cell i
M, N_{eff}	particle count and effective sample size

It needs a TeX distribution with the `amsmath`, `siunitx`, `booktabs`, `tikz` and `listings` packages (on Ubuntu: `texlive-latex-extra` `texlive-science`).

Bibliography

- [1] J. M. Stumfoll, “Discrete-time neural network based state observer with neural network based control formulation for a class of systems with unmatched uncertainties,” m.s. thesis, revision 8, Missouri University of Science and Technology, n.d. Source: <https://github.com/jstumfoll2/twip-dmso>, docs/thesis.
- [2] T. Foote and M. Purvis, “REP 103: Standard units of measure and coordinate conventions.” <https://www.ros.org/reps/rep-0103.html>, 2010.
- [3] W. Meeussen, “REP 105: Coordinate frames for mobile platforms.” <https://www.ros.org/reps/rep-0105.html>, 2010.
- [4] A. Mandow, J. L. Martínez, J. Morales, J. L. Blanco, A. García-Cerezo, and J. González, “Experimental kinematics for wheeled skid-steer mobile robots,” *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 1222–1227, 2007.
- [5] J. L. Martínez, A. Mandow, J. Morales, S. Pedraza, and A. García-Cerezo, “Approximating kinematics for tracked mobile robots,” *The International Journal of Robotics Research*, vol. 24, no. 10, pp. 867–878, 2005.
- [6] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.
- [7] J. Y. Wong, *Theory of Ground Vehicles*. Hoboken, NJ: John Wiley and Sons, 4th ed., 2008.
- [8] D. Gebre-Egziabher, *Design and Performance Analysis of a Low-Cost Aided Dead Reckoning Navigator*. PhD thesis, Stanford University, 2004.
- [9] W. S. Flenniken, “Modeling inertial measurement units and analyzing the effect of their errors in navigation applications,” Master’s thesis, Auburn University, 2005.
- [10] P. D. Groves, *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*. Boston, MA: Artech House, 2nd ed., 2013.
- [11] J. B. Kuipers, *Quaternions and Rotation Sequences*. Princeton, NJ: Princeton University Press, 1999.

- [12] D. Gebre-Egziabher, G. H. Elkaim, J. D. Powell, and B. W. Parkinson, "Calibration of strapdown magnetometers in magnetic field domain," *Journal of Aerospace Engineering*, vol. 19, no. 2, pp. 87–102, 2006.
- [13] I. Kåsa, "A circle fitting procedure and its error analysis," *IEEE Transactions on Instrumentation and Measurement*, vol. IM-25, no. 1, pp. 8–14, 1976.
- [14] NOAA National Centers for Environmental Information, "The world magnetic model, WMM2025." <https://www.ncei.noaa.gov/products/world-magnetic-model>, 2025.
- [15] W. T. Higgins, "A comparison of complementary and Kalman filtering," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 11, no. 3, pp. 321–325, 1975.
- [16] P. Misra and P. Enge, *Global Positioning System: Signals, Measurements, and Performance*. Lincoln, MA: Ganga-Jamuna Press, 2nd ed., 2006.
- [17] T. Moore and D. Stouch, "A generalized extended Kalman filter implementation for the Robot Operating System," in *Proceedings of the 13th International Conference on Intelligent Autonomous Systems (IAS-13)*, pp. 335–348, Springer, 2014.
- [18] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Transactions of the ASME, Journal of Basic Engineering*, vol. 82, pp. 35–45, 1960.
- [19] D. Simon, *Optimal State Estimation*. Hoboken, NJ: John Wiley and Sons, 2006.
- [20] S. Umeyama, "Least-squares estimation of transformation parameters between two point patterns," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.
- [21] G. Grisetti, C. Stachniss, and W. Burgard, "Improved techniques for grid mapping with Rao-Blackwellized particle filters," *IEEE Transactions on Robotics*, vol. 23, no. 1, pp. 34–46, 2007.
- [22] A. Doucet, N. de Freitas, K. Murphy, and S. Russell, "Rao-Blackwellised particle filtering for dynamic Bayesian networks," in *Proceedings of the Sixteenth Conference on Uncertainty in Artificial Intelligence*, pp. 176–183, 2000.
- [23] K. Murphy, "Bayesian map learning in dynamic environments," in *Advances in Neural Information Processing Systems 12*, pp. 1015–1021, 1999.
- [24] S. Kohlbrecher, O. von Stryk, J. Meyer, and U. Klingauf, "A flexible and scalable SLAM system with full 3D motion estimation," in *IEEE International Symposium on Safety, Security, and Rescue Robotics*, pp. 155–160, 2011.